Software Engineering & Project Management (SEPM)

Energy & Utility Management System (EUMS)

Electricity · Water · Gas — billing, metering and customer service

Mid-Semester Project Report

Software process model: Waterfall · Combined SEPM + DBMS course project

Group number	Group __
Course and section	Software Engineering & Project Management (SEPM), Section __
Supervisor / instructor	Prof. _____
Institute	_____ Institute of Technology
Date	October 2026

#	Name	Registration number
1	Sushrut Lad	_____
2	Member 2	_____
3	Member 3	_____
4	Member 4	_____

Contents

1. Problem statement and motivation

2. Project objectives

3. Software process model (Waterfall) and feasibility study

4. Scope of the project

5. Stakeholder identification

6. Requirements specification (functional, non-functional, prioritisation)

7. Requirements model (use cases and user stories)

8. System design (architecture, modules, database, UML)

9. Technology stack and justification

10. Implementation progress

11. Testing performed

12. Project-management status (plan, progress, risks, estimation)

13. Individual contribution

14. Challenges faced and solutions

15. Work planned for the remaining semester

16. References

Appendix A – AI-tool usage disclosure · Appendix B – Screenshots · Appendix C – How to run

> **Demo video:** PASTE-VIEW-ONLY-LINK-HERE
> **Supporting files:** SRS, use cases, design diagrams, test cases and project-management records are also provided as separate PDFs in folders 02–06 of this ZIP. The full source code is in `04_Source_Code/`.

1. Problem Statement and Motivation

Small and mid-sized utility providers and housing-society utility offices in India still run electricity, water and piped-gas services with paper meter cards, spreadsheets and counter-based payment. Field staff write readings in registers, office staff re-type them, and bills are worked out by hand from tariff tables that change every year. This causes:

- **Billing errors** – progressive (slab) tariffs are easy to calculate wrongly by hand; a single mistyped reading produces a wrong or even negative bill.
- **No single view of a customer** – electricity, water and gas records live in different files, so the office cannot see a customer's total dues or history in one place.
- **Slow payments and long queues** – customers must visit a counter, and payment status is updated late.
- **Poor complaint and outage handling** – complaints are lost, and customers are not told about outages in their area.
- **No management information** – managers cannot easily see revenue, pending dues, consumption by area or overdue bills.

Problem statement. *Design and build a centralised, database-driven web application that manages customers, connections, meters, meter readings, slab-based billing, payments, complaints and outages for electricity, water and gas, with separate role-based views for administrators, employees and customers.*

Motivation. The topic was assigned for our DBMS course project. Because it is a realistic multi-user system with clear business rules, we chose to build the same system for the SEPM course as well, following the Waterfall model. The DBMS course focuses on the schema, constraints, triggers and views; this SEPM report focuses on the engineering process: feasibility, requirements, design, implementation, testing and project management.

2. Project Objectives

1. **O1 – Digitise metering:** let employees record meter readings in a web form that shows the previous reading and calculates consumption immediately, and that rejects impossible readings.
2. **O2 – Accurate automatic billing:** generate a bill automatically from each reading using configurable slab tariffs, fixed charges and tax, with an itemised breakdown.
3. **O3 – Self-service for customers:** customers can view their connections, bills and usage history, pay online (simulated gateway) and raise complaints.
4. **O4 – Service management:** record and track complaints and outages; notify affected customers automatically.
5. **O5 – Management dashboard:** show KPIs, revenue by utility, pending dues, overdue bills and area-wise consumption.
6. **O6 – Data integrity and security:** enforce business rules in the database (constraints and triggers), hash passwords, and restrict every page by role.
7. **O7 – Apply SEPM practice:** follow the Waterfall model with documented feasibility study, SRS, design, test cases and project plan.

3. Software Process Model and Feasibility Study

3.1 Choice of process model: Waterfall

We follow the classical **Waterfall model**: each phase produces a document that is reviewed and frozen before the next phase starts. Waterfall suits this project because:

- The requirements are **well understood and stable**: the domain, metering, slab billing and payments, follows published tariff rules and the scope was fixed by the course.
- The DBMS course needs a **complete database design up front** (ER model, normalisation), which matches Waterfall's design-before-code approach.
- The course has **fixed milestones** (mid-semester and end-semester), which map naturally onto phase completion.
- The team is new to web development, so a **clear, documented sequence** of phases is easier to manage than iterative sprints.

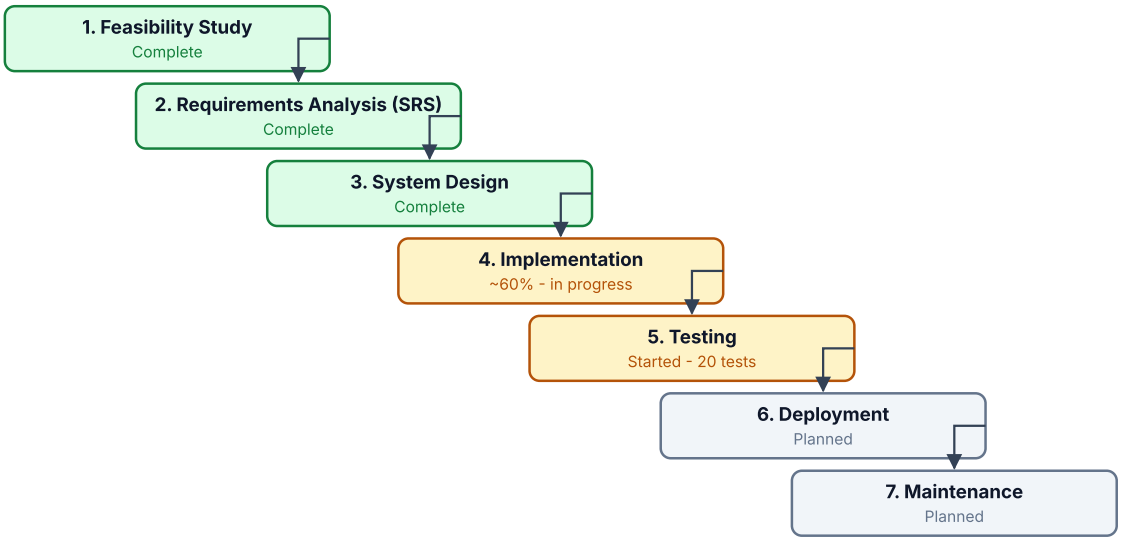


Figure 1: Waterfall phases and their status at mid-semester

Phase	Output (deliverable)	Status at mid-sem
1. Feasibility study	Feasibility report (section 3.2)	Complete
2. Requirements analysis	SRS: functional / non-functional requirements, use cases (sections 6–7, SRS.pdf)	Complete
3. System design	Architecture, ER model, schema, UML, UI design (section 8)	Complete
4. Implementation	Working Flask + SQLite application (section 10)	About 60% complete
5. Testing	Unit and functional test suite (section 11)	Started: 20 automated tests
6. Deployment	Installed system, user guide	Planned
7. Maintenance	Bug fixes, enhancements	Planned

Table 1: Waterfall phases, deliverables and status

Because pure Waterfall makes it costly to find mistakes late, we added two small safeguards: each phase ends with a team review against the previous phase's document, and unit tests were written alongside implementation instead of only at the end. The overall flow remains sequential.

3.2 Feasibility study

Technical feasibility

The system needs only a web server, a relational database and a browser. We use Python with the Flask framework and SQLite, which are free, run on any laptop (Windows, Linux or macOS) and need no separate database server. SQLite supports everything the DBMS part needs: primary/foreign keys, CHECK and UNIQUE constraints, triggers, views and indexes. Team members already know Python and SQL from earlier courses; Flask and HTML templates can be learned in a few days. **Conclusion: technically feasible.**

Economic feasibility

Item	Cost	Remark
Software (Python, Flask, SQLite, VS Code, Git/GitHub, Chart.js)	₹0	All open source / free
Hardware	₹0	Team members' existing laptops
Hosting for demo (optional)	₹0	Runs locally; free tiers exist if needed
Payment gateway	₹0	Simulated in this version; real gateways charge per transaction
Effort	4 students × ~6 h/week × 14 weeks	About 336 person-hours

Table 2: Economic feasibility (development cost)

For a real utility office, the main benefits are fewer billing errors, faster collection and less manual data entry, at negligible software cost. **Conclusion: economically feasible.**

Operational feasibility

The users are office staff, field meter readers and customers, all of whom already use smartphones and UPI apps. The interface uses simple forms, one page per task, with the same layout as common billing portals. Validation messages are shown in plain language (for example "Reading cannot be lower than the previous reading"). Little training is needed. **Conclusion: operationally feasible.**

Schedule feasibility

The semester has about 14 working weeks. The plan (section 12) gives roughly 4 weeks to feasibility, requirements and design, 6 weeks to implementation and 4 weeks to testing, deployment and documentation. Core features were finished by mid-semester, so the schedule is realistic. **Conclusion: feasible within the semester.**

Legal and ethical feasibility

Only sample (fictional) customer data is used. Passwords are stored as salted hashes, and customers can see only their own data. A production version would need to follow the Digital Personal Data Protection Act, 2023 (consent and data minimisation) and RBI/PCI rules for payment data; our simulated gateway stores no card or UPI details. **Conclusion: feasible for an academic prototype.**

4. Scope of the Project

4.1 Features included

Module	Included functionality
Authentication & roles	Login/logout, hashed passwords, three roles (Admin, Employee, Customer), role check on every page
Customer management	Register customer (creates login), search, customer profile with connections and bills
Connections & meters	Electricity / water / gas connection per customer, automatic tariff selection, meter creation
Meter readings	Reading entry with live previous-reading lookup and consumption; monotonic and date-order validation
Billing	Automatic slab-based bill generation, itemised bill, printable bill (save as PDF from browser)
Payments	Online payment (card / UPI / net banking / wallet, simulated), counter cash payment, partial payment, receipts
Complaints	Customer raises complaint; staff assign, prioritise, update and resolve
Outages	Report scheduled/active outage by area; affected customers notified automatically; mark resolved
Tariffs	Admin edits slab rates, fixed charge and tax; old bills unaffected
Dashboards & reports	Admin KPIs and charts, customer dashboard, usage history, area-wise and category-wise analytics, overdue bills
Notifications	In-app notifications for new bills, payments, outages and complaint updates

Table 3: Features in scope

4.2 Features excluded

- Real payment-gateway integration (Razorpay/PayU) – payments are simulated.
- Smart-meter / IoT automatic reading – readings are entered manually.
- SMS and e-mail delivery – notifications are in-app only.
- Mobile app – the web pages are responsive instead.
- Multiple tariff versions with effective dates, late-payment penalties, GST invoices and accounting export.
- Multi-company (multi-tenant) support.

4.3 Assumptions and constraints

- Each customer has at most one connection per utility; each connection has one active meter.
- Meters are cumulative, so readings only increase; meter replacement is handled by adding a new meter.
- Customers are either residential or commercial; the tariff is chosen from the utility and category.
- One bill is generated per reading, normally once a month; the due date is 15 days after the bill date.

- Constraint: the system must run on a student laptop with free software and no database server.
- Constraint: development time is one semester with four part-time developers.

5. Stakeholder Identification

Stakeholder	Type	Interest / needs	Interaction with system
Administrator (utility manager)	Primary user	Control tariffs and staff; see revenue, dues and service KPIs	Dashboard, tariffs, employees, reports, outages
Meter reader	Primary user	Fast, error-free reading entry in the field	Add Meter Reading page
Billing officer	Primary user	Correct bills, record counter payments, register customers	Customers, connections, billing, payments
Technician	Primary user	See assigned complaints and outages, update status	Complaints, outages
Customer (residential / commercial)	Primary user	See and pay bills, track usage, report problems, know about outages	Customer portal
Course instructor / evaluator	Secondary	Evidence of SEPM and DBMS practice, working demo	Reports, demo, source code
Development team	Secondary	Deliver on time, learn the stack	Builds and tests the system
Regulator (e.g. State Electricity Regulatory Commission)	External	Tariffs applied correctly; bills auditable	Indirect – tariff rules, bill snapshot
Payment provider (bank / UPI)	External	Secure payment handling	Simulated in this version

Table 4: Stakeholders

6. Requirements Specification

Requirements were gathered from (a) the DBMS project brief, (b) study of real electricity/water bills and published slab tariffs, (c) existing utility self-service portals, and (d) team brainstorming sessions with role-play of each user type. The full IEEE-830-style SRS is provided separately in `02_Requirements/SRS.pdf`; the key requirements are summarised here.

6.1 Functional requirements

ID	Module	Requirement	Priority	Status
FR-01	Authentication	Users log in with username/email and password; passwords are stored as salted hashes; users can log out.	Must	Implemented

FR-02	Authorisation	Each page is restricted to its roles (Admin, Employee, Customer). Unauthorised access returns HTTP 403; not logged in redirects to login.	Must	Implemented
FR-03	Customers	Staff register a customer (name, email, 10-digit phone, address, area, category, initial password). Email must be unique. A customer code (C1001…) is generated.	Must	Implemented
FR-04	Customers	Staff search customers by name, code, area or phone and open a customer profile showing connections and recent bills.	Should	Implemented
FR-05	Connections	Staff create a connection for a customer and utility; the tariff is chosen from the customer category; a meter with a starting reading is created. Only one connection per utility per customer.	Must	Implemented
FR-06	Meter readings	Employee selects a meter; the system shows customer, utility and previous reading, and calculates consumption as the current value is typed.	Must	Implemented
FR-07	Meter readings	A reading lower than the previous reading, or dated on/before the last reading, is rejected.	Must	Implemented
FR-08	Billing	Saving a reading generates a bill automatically: slab charges + fixed charge + tax, with itemised rows, bill number and due date (+15 days).	Must	Implemented
FR-09	Billing	Staff list bills filtered by status and search; any user can open an allowed bill and print / save it as PDF.	Must	Implemented
FR-10	Payments	Customer pays a bill online by card, UPI, net banking or wallet (simulated); partial payments allowed; a receipt number is generated.	Must	Implemented
FR-11	Payments	Payment larger than the outstanding amount is rejected. Bill status changes to partial or paid automatically.	Must	Implemented
FR-12	Payments	Staff record counter (cash) payments and view all payments with totals by method.	Should	Implemented
FR-13	Complaints	Customer raises a complaint for a connection with urgency; staff view by status, assign to an employee, change priority/status and add resolution notes.	Must	Implemented
FR-14	Outages	Staff report an outage (utility, area, reason, start, estimated end); future outages are "scheduled". Staff mark outages resolved.	Should	Implemented
FR-15	Notifications	Customers are notified in-app of new bills, payments, outages in their area for their utilities, and complaint status changes.	Should	Implemented
FR-16	Tariffs	Admin edits slab rates, fixed charge and tax per plan; changes apply only to new bills.	Must	Implemented
FR-17	Employees	Admin views employees with workload and adds new employees.	Should	Implemented

FR-18	Dashboards	Admin/employee dashboard shows KPIs, billed amount per utility (6 months), collections, recent activity. Customer dashboard shows connections, current bills, monthly bill chart and recent bills.	Should	Implemented
FR-19	Reports	Admin sees usage per utility, area-wise consumption, billed amount by category, collection efficiency and overdue bills.	Could	Partial
FR-20	Customer portal	Customer views connections, usage history (per-utility charts), outages in their area and edits phone/address and password.	Should	Implemented
FR-21	Billing	Late-payment surcharge and automatic overdue reminders.	Could	Planned
FR-22	Billing	Downloadable server-generated PDF invoice and e-mail delivery.	Could	Planned
FR-23	Meters	Meter replacement workflow (mark faulty/replaced, new meter with baseline reading).	Should	Planned
FR-24	Data export	Export reports to CSV/Excel.	Could	Planned
FR-25	Payments	Real payment-gateway integration.	Won't (now)	Planned

Table 5: Functional requirements with MoSCoW priority and status

6.2 Non-functional requirements

ID	Category	Requirement	How it is met / measured
NFR-01	Security	Passwords never stored in plain text.	Werkzeug salted hash (scrypt/PBKDF2)
NFR-02	Security / privacy	A customer can access only their own records.	Every portal query filtered by logged-in customer; 403 otherwise (test IT-05)
NFR-03	Security	No secrets in source code.	`config.py` is git-ignored; only `config.example.py` is committed
NFR-04	Data integrity	Business rules hold even if application code has a bug.	CHECK, UNIQUE, FK constraints and 7 triggers in the database
NFR-05	Accuracy	Bills are correct to the paisa.	Amounts rounded to 2 decimals; unit tests UT-01..06
NFR-06	Auditability	Issued bills never change when tariffs change.	Charges stored on the bill (snapshot); test IT-14
NFR-07	Performance	Pages load in under 1 second for 1,000 customers on a laptop.	Indexes on readings, bills, customers, complaints, notifications

NFR-08	Usability	A new employee can enter a reading without training.	One task per page, live consumption, plain-language errors
NFR-09	Portability	Runs on Windows, Linux, macOS with Python 3.10+.	Only Flask needed; SQLite is built into Python
NFR-10	Responsiveness	Usable on a phone screen (meter readers in the field).	Responsive CSS layout below 800 px
NFR-11	Maintainability	Code is easy for students to read and change.	Small files by role; billing logic separate from web code; automated tests
NFR-12	Reliability	A failed operation leaves no half-saved data.	Each action in one transaction; rollback on error

Table 6: Non-functional requirements

6.3 Prioritisation of requirements

We used the **MoSCoW** method. Must requirements make up the minimum system that solves the core problem (readings → bills → payments, plus access control and complaints). Should requirements add important value but the system still works without them. Could items are added if time allows, and Won't (now) items are out of scope for this semester. Implementation followed this order, so every Must requirement was completed before mid-semester.

Priority	Count	Implemented	Partial	Planned
Must	12	12	0	0
Should	8	7	0	1
Could	4	0	1	3
Won't (now)	1	0	0	1
Total	**25**	**19**	**1**	**5**

Table 7: Requirement status by priority

7. Requirements Model: Use Cases and User Stories

7.1 Use-case diagram

Three actors use the system. "Employee" covers the meter reader, billing officer and technician, who share one role in the system. UC-05 *Generate bill* is never started by a person; it is included in UC-04, so every saved reading produces a bill.

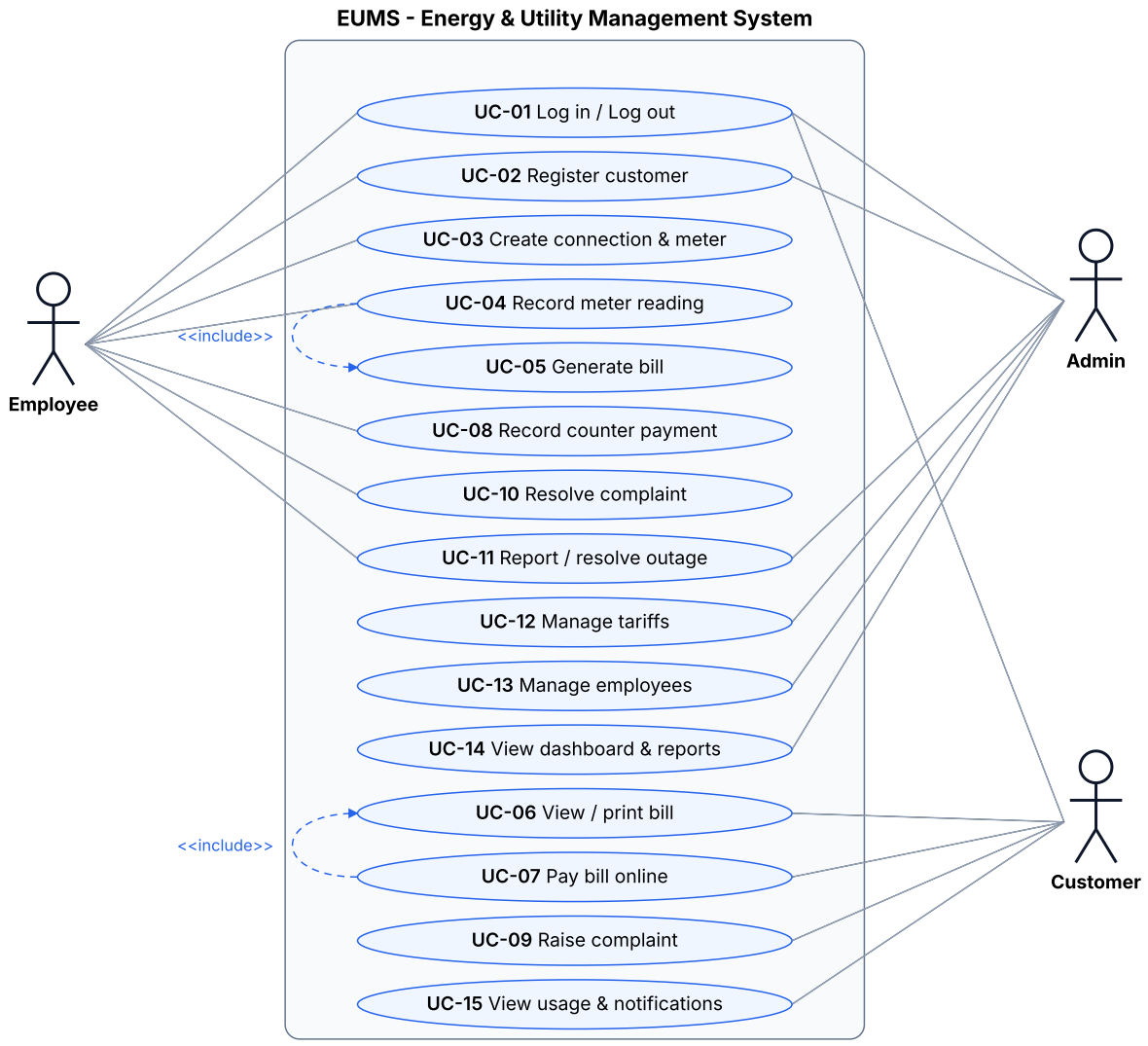


Figure 2: Use-case diagram of EUMS

7.2 Use-case descriptions

UC-04 Record meter reading (includes UC-05 Generate bill)	
Actor(s)	Employee
Pre-condition	Employee is logged in; the connection and meter are active.
Main flow	1. Employee opens Meter Readings → Add Reading. 2. Employee selects a meter; system shows customer name, utility and previous reading with its date. 3. Employee types the current reading; system shows the consumption immediately. 4. Employee enters the reading date and optional remarks and clicks Save Reading. 5. System saves the reading, calculates the bill from the tariff slabs (UC-05), saves the bill and its items, and notifies the customer. 6. System opens the new bill.

Alternate / exception flows	• 3a. Current reading lower than previous: an error is shown and Save is disabled. If submitted anyway, the database trigger rejects it and nothing is saved. • 4a. Date on or before the last reading date: the trigger rejects it and an error is shown. • 4b. Reading already exists for that date: error shown.
Post-condition	Reading and bill stored; customer sees the bill and a "New bill generated" notification.

UC-07 Pay bill online	
Actor(s)	Customer
Pre-condition	Customer is logged in and has a bill that is unpaid or partial.
Main flow	1. Customer opens Bills & Payments and clicks Pay on a bill. 2. System shows bill details and the outstanding amount, pre-filled. 3. Customer optionally changes the amount (partial payment) and chooses card / UPI / net banking / wallet. 4. Customer clicks Pay. 5. System records the payment with a receipt number; trigger updates bill to paid or partial and notifies the customer. 6. System shows the bill with the payment and a success message.
Alternate / exception flows	• 1a. Bill belongs to another customer: HTTP 403. • 4a. Amount more than outstanding: trigger rejects; error shown; nothing saved. • 4b. No method chosen: error shown. • 1b. Bill already paid: message "already paid".
Post-condition	Payment saved; bill status updated; receipt number shown.

Descriptions of UC-09/10, UC-11 and UC-01 are in `02_Requirements/Use_Cases_or_User_Stories.pdf`.

7.3 User stories with acceptance criteria

ID	User story	Acceptance criteria	Status
US-01	As a **meter reader**, I want to see the previous reading when I pick a meter, so that I do not mistype the new one.	Given a meter is selected, previous value and date are shown; consumption updates while typing; lower values show an error and disable Save.	Done
US-02	As a **billing officer**, I want bills to be calculated automatically from slabs, so that there are no manual errors.	For 250 kWh on the domestic plan the bill is ₹1,207.50 with two slab rows, fixed charge and 5% tax.	Done
US-03	As a **customer**, I want to pay my bill online by UPI or card, so that I do not have to visit the office.	Pay page shows outstanding amount; payment creates receipt RCPyyyymmdd-nnnn; bill becomes Paid; overpayment rejected.	Done

US-04	As a **customer**, I want to see all my connections and current bills on one dashboard.	Dashboard shows one card per connection with latest bill amount, due date and status, plus a 6-month chart.	Done
US-05	As a **customer**, I want to be told about outages in my area.	When staff report an outage for area A and utility U, all customers in A with U get a notification; others do not.	Done
US-06	As a **technician**, I want to see open complaints and update them.	Complaints page has tabs All/Open/In progress/Resolved with counts; saving Resolved records the time and notifies the customer.	Done
US-07	As an **admin**, I want to change tariff rates without changing old bills.	Edited rates are used for the next bill; previously issued bill totals are unchanged.	Done
US-08	As an **admin**, I want to see revenue, dues and complaints at a glance.	Dashboard shows 6 KPI cards, billed amount by utility for 6 months, collections split and recent activity.	Done
US-09	As an **admin**, I want overdue reminders and late fees to be applied automatically.	Bills unpaid after the due date get a surcharge and the customer is reminded.	Planned
US-10	As a **billing officer**, I want to replace a faulty meter without breaking reading history.	Old meter marked Replaced; new meter starts at its own baseline; next bill uses the new meter.	Planned

Table 8: User stories

8. System Design

The design was completed and reviewed before coding started (Waterfall phase 3). It covers the architecture, the modules, the database and the main behaviour (UML).

8.1 Architecture diagram

EUMS uses a **three-tier (layered) client–server architecture** with the Model-View-Controller idea: templates are the *view*, Flask route functions are the *controller*, and the SQLite tables with `billing.py` form the *model*. The browser sends ordinary HTTP GET/POST requests (HTML forms); the server returns finished HTML pages. One small JSON endpoint gives the previous meter reading to the reading form.

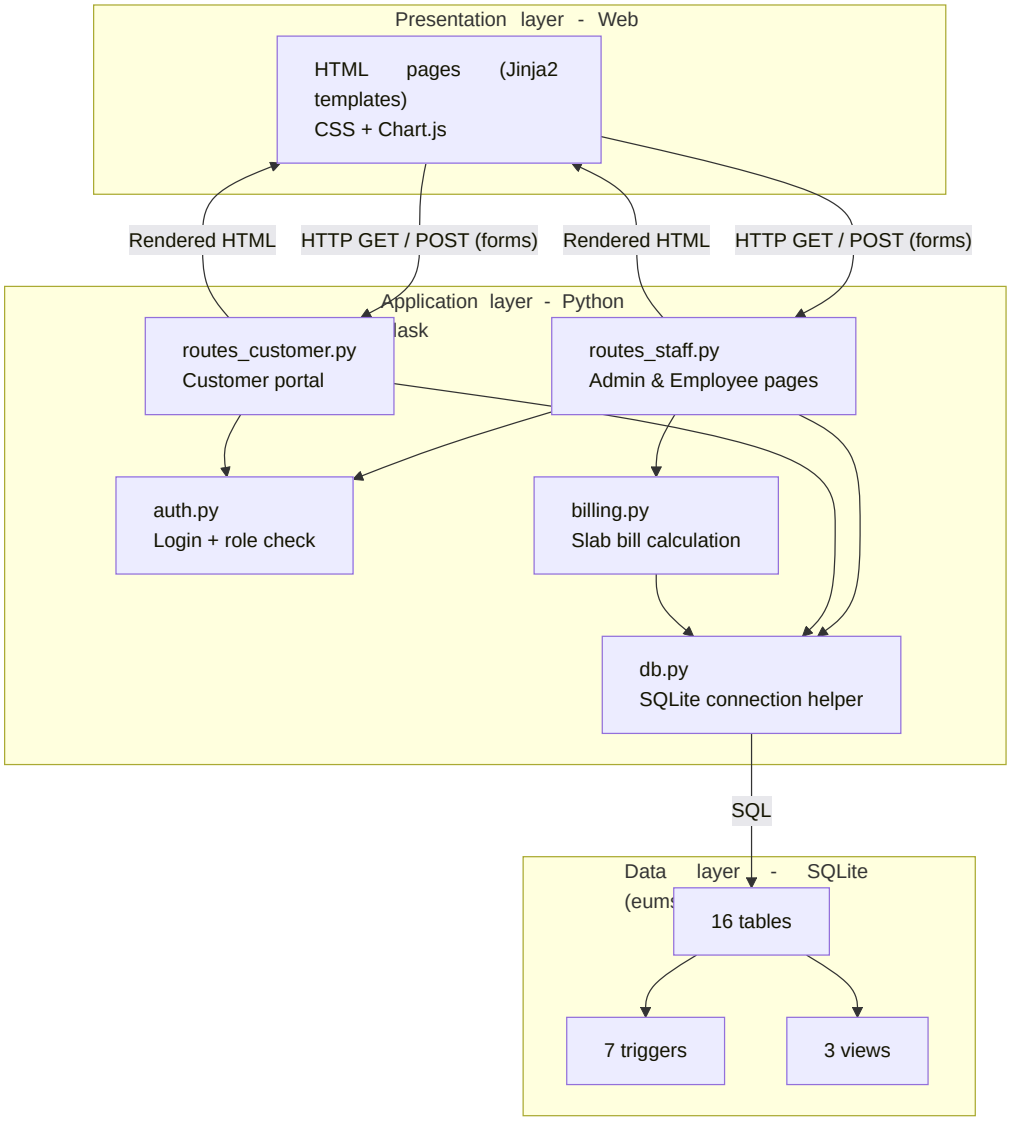


Figure 3: Three-tier architecture of EUMS

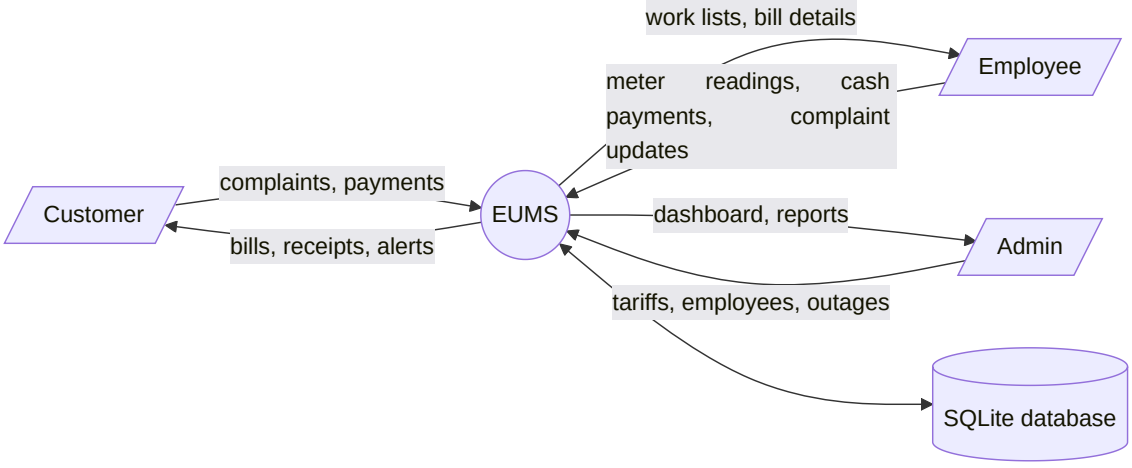


Figure 4: Context-level (Level 0) data flow diagram

8.2 Module description

File / module	Responsibility

`app.py`	Creates the Flask app, loads configuration, registers the two route groups, login/logout pages, error pages, and template helpers (money, date formatting).
`auth.py`	`login_required(*roles)` decorator: redirects to login if not signed in, returns 403 if the role is not allowed.
`db.py`	Opens one SQLite connection per request (foreign keys on, rows as dictionaries) and closes it afterwards; `query()` helper.
`billing.py`	Pure function `calculate_bill()` for slab pricing (unit tested) and `generate_bill()`, which reads the tariff and inserts the bill and bill items.
`routes_staff.py`	Admin and employee pages: dashboard, customers, connections, meter readings, billing, payments, complaints, outages, tariffs, employees, reports.
`routes_customer.py`	Customer portal: dashboard, connections, bills, pay, usage history, complaints, outages, profile, notifications. Every query is filtered by the logged-in customer.
`schema.sql`	All tables, constraints, indexes, triggers and views.
`seed.py`	Creates the database and loads demo data (24 customers from `sample_data/customers.csv`, 6 months of readings, bills, payments, complaints, outages).
`templates/`, `static/`	Jinja2 HTML pages (one per feature), one CSS file, and a small Chart.js helper.
`tests/`	pytest unit tests (billing) and functional tests (whole app with a fresh database per test).

Table 9: Modules

8.3 ER diagram and data model

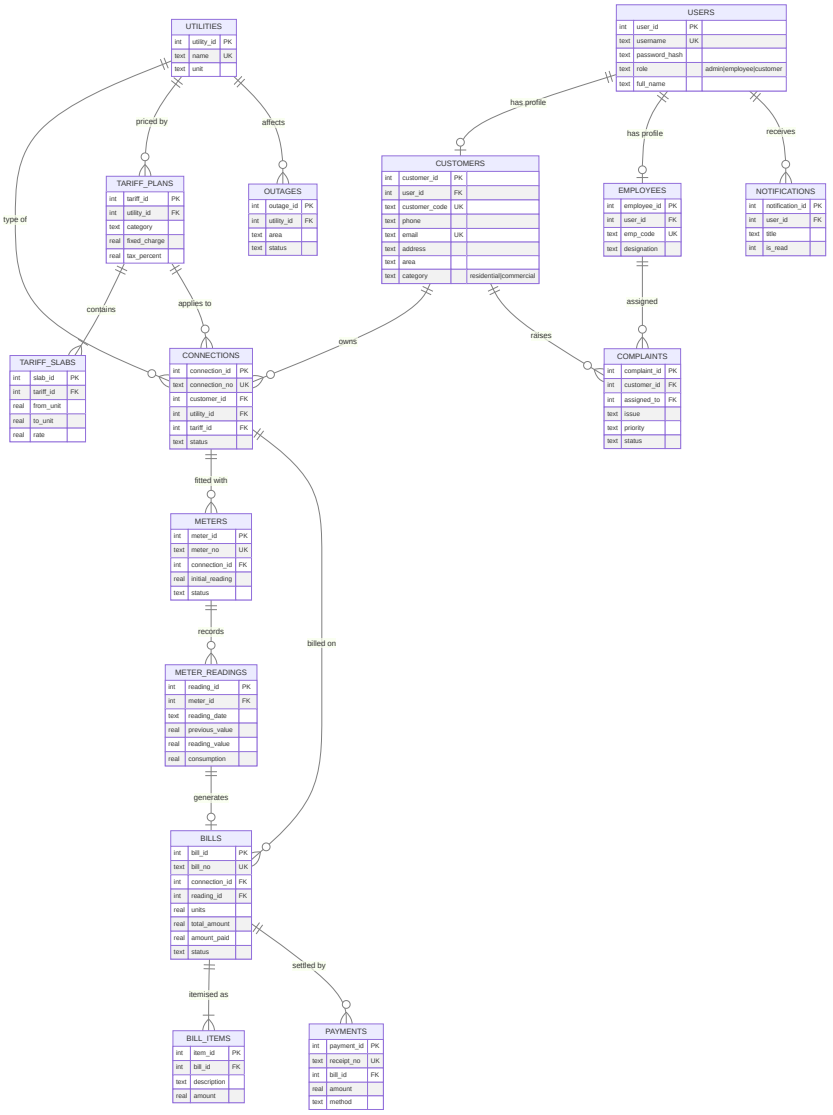


Figure 5: Entity–relationship diagram (crow's-foot notation)

Table	Purpose	Keys and important constraints
users	Login account for every person	PK user_id; UNIQUE username; CHECK role IN (admin, employee, customer)
customers	Customer profile	PK customer_id; FK user_id UNIQUE (1:1); UNIQUE customer_code, email; CHECK category
employees	Staff profile	PK employee_id; FK user_id UNIQUE; UNIQUE emp_code
utilities	Electricity / Water / Gas and their units	PK utility_id; UNIQUE name
tariff_plans	Fixed charge and tax for a utility + category	PK tariff_id; FK utility_id; UNIQUE (utility_id, category)

tariff_slabs	Rate per unit range	PK slab_id; FK tariff_id ON DELETE CASCADE; CHECK to_unit > from_unit
connections	A customer's service for one utility	PK; FK customer, utility, tariff; UNIQUE (customer_id, utility_id)
meters	Physical meter on a connection	PK; UNIQUE meter_no; CHECK status; initial_reading ≥ 0
meter_readings	Cumulative reading on a date	PK; FK meter_id; UNIQUE (meter_id, reading_date); CHECK consumption ≥ 0
bills	Bill for one reading (snapshot of charges)	PK; UNIQUE bill_no, reading_id (1:1 with reading); CHECK status
bill_items	Itemised lines of a bill	PK; FK bill_id ON DELETE CASCADE
payments	Money received against a bill	PK; UNIQUE receipt_no; CHECK amount > 0; CHECK method
complaints	Customer complaints	PK; FK customer, connection, assigned employee; CHECK priority, status
outages	Supply interruptions by area	PK; FK utility_id; CHECK status
notifications	In-app messages to a user	PK; FK user_id; index (user_id, is_read)
audit_log	Recent activity feed for the dashboard	PK log_id

Table 10: Tables in the schema (16)

8.4 Normalisation

- **1NF:** every column holds one atomic value; repeating data (many readings, many payments, many slabs) is in its own table, not in repeated columns.
- **2NF:** every table has a single-column surrogate primary key, so there are no partial dependencies. For natural composite keys such as (meter_id, reading_date), the non-key columns depend on the whole key.
- **3NF / BCNF:** there are no transitive dependencies. For example, tariff rates are not stored in `connections`; a connection points to a tariff plan and the plan points to its slabs. Utility units live only in `utilities`.
- **Deliberate denormalisation:** `bills` stores units, energy charge, fixed charge, tax and total at the time the bill is issued. Tariffs change over time, but an issued bill is a legal record and must never change. `meter_readings.consumption` is also stored so that a reading row is self-explanatory and cannot drift.

8.5 Triggers, views and indexes

Object	Type	Rule enforced

trg_reading_monotonic	BEFORE INSERT	Rejects a reading lower than the meter's highest reading (or initial reading).
trg_reading_date_order	BEFORE INSERT	Rejects a reading dated on/before the last reading date.
trg_no_overpay	BEFORE INSERT	Rejects a payment larger than the bill's outstanding amount.
trg_payment_status	AFTER INSERT	Adds to amount_paid, sets status paid/partial, notifies customer, logs activity.
trg_bill_notify	AFTER INSERT	Notifies the customer when a bill is generated.
trg_outage_notify	AFTER INSERT	Notifies every customer in the outage area who has that utility.
trg_complaint_status	AFTER UPDATE	Notifies the customer when complaint status changes.
v_bill_summary	View	Bill joined with connection, utility and customer, with outstanding amount.
v_overdue_bills	View	Unpaid bills past due date, with days overdue.
v_monthly_consumption	View	Total units and billed amount per utility per month (dashboard and reports).
6 indexes	B-tree	(meter_id, reading_date), (status, due_date), billing_month, area, complaint status, (user_id, is_read).

Table 11: Database objects that enforce business rules

Example – the overpayment trigger (from `schema.sql`):

```
CREATE TRIGGER trg_no_overpay
BEFORE INSERT ON payments
WHEN NEW.amount > (SELECT ROUND(total_amount - amount_paid, 2) FROM bills WHERE bill_id =
NEW.bill_id) + 0.001
BEGIN
    SELECT RAISE(ABORT, 'Payment exceeds outstanding balance');
END;
```

8.6 Billing formula

For each slab *i* with range [from_i, to_i) and rate r_i (the last slab has no upper limit):

$$\text{units in slab}_i = \max(0, \min(U, \text{to}_i) - \text{from}_i) \qquad \text{Energy} = \Sigma\ \text{units in slab}_i \times r_i \qquad \text{Total} = (\text{Energy} + \text{Fixed}) \times (1 + \text{Tax\%} / 100)$$

Worked example (domestic electricity, 250 kWh): 100 × 3.50 = 350.00; 150 × 5.00 = 750.00; energy = 1,100.00; + fixed 50.00 = 1,150.00; + 5% tax 57.50 = **₹1,207.50** (verified by test UT-01).

8.7 Sequence diagram – meter reading to bill to payment

This is the main end-to-end flow and the one shown in the demo video.

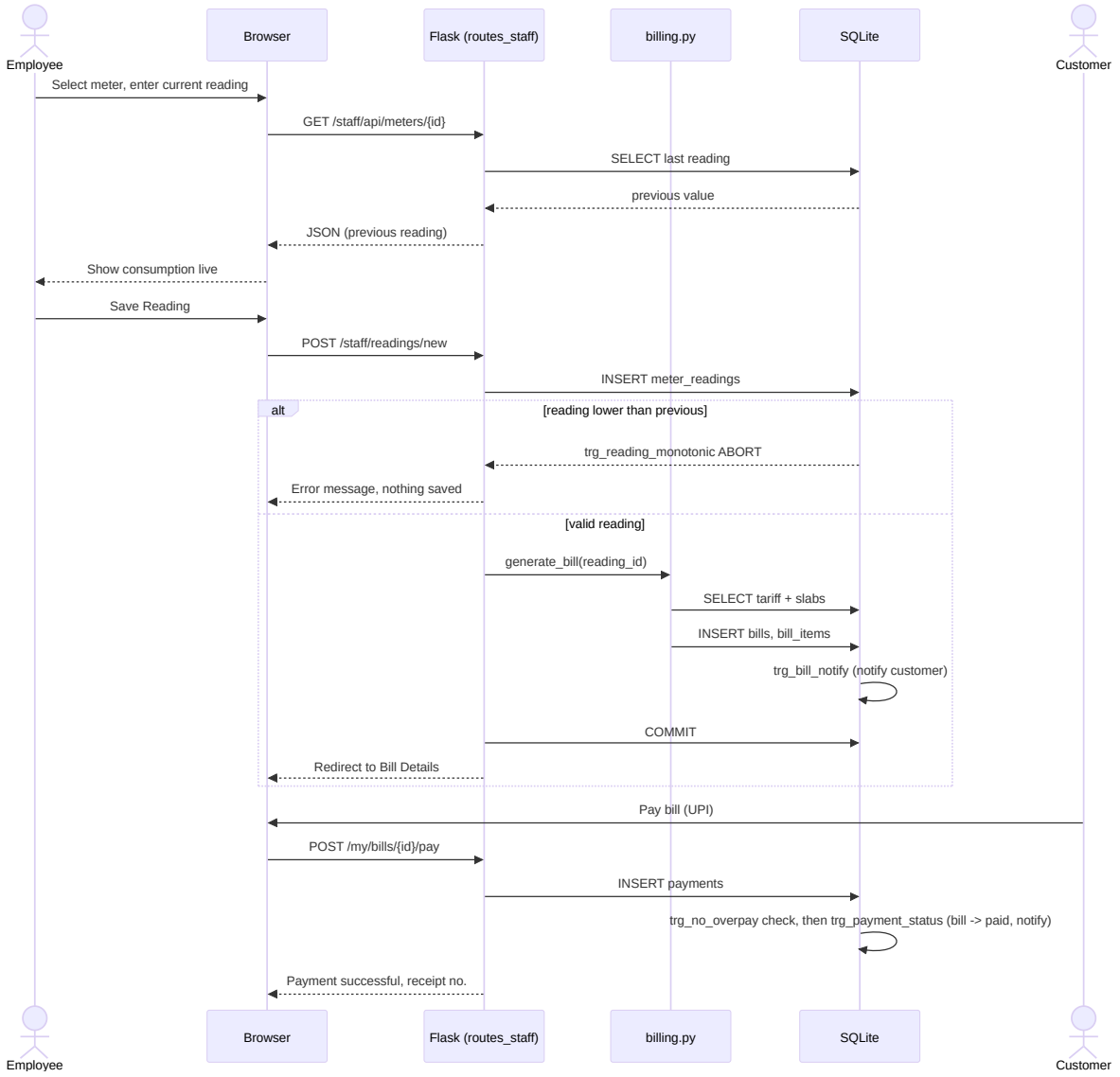


Figure 6: Sequence diagram: record reading, generate bill, pay bill

8.8 Activity diagram – online bill payment

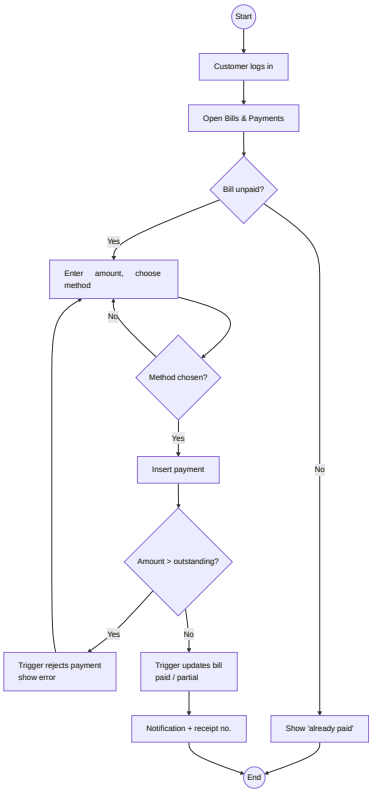


Figure 7: Activity diagram for paying a bill

8.9 Class diagram (domain model)

The application is written in a procedural Flask style, so these classes are the *domain model*: each class is a database table, and the operations are carried out by route functions and `billing.py`.

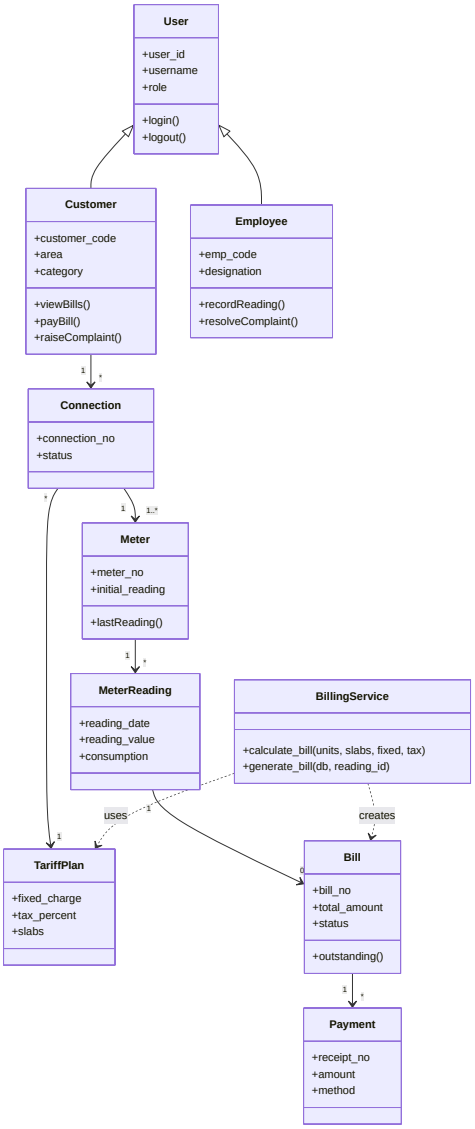


Figure 8: Class diagram of the main domain entities

8.10 User-interface design

The UI was designed from a reference mock-up (UtiliPro style: dark navy sidebar, white cards, blue accents) and implemented as one page per feature. Utility colours are fixed everywhere: electricity = blue, water = green, gas = orange. Screenshots of every page are in Appendix B and in `03_Design/UI_Screens.pdf`.

9. Technology Stack and Justification

We deliberately chose the **simplest stack that still meets all requirements**, so every team member can read, run and explain all of the code.

Layer	Technology	Why we chose it	Alternatives considered
Language	Python 3.10+	Already taught in earlier semesters; readable; large ecosystem.	Java, PHP

Web framework	Flask 3	Very small: a page is one function with a URL. No build step. Includes password hashing (Werkzeug) and templates (Jinja2).	Django (too much to learn), Node/Express
Database	SQLite 3	Built into Python, single file, zero setup, but still supports constraints, triggers, views and indexes for the DBMS course. Schema is standard SQL and can be moved to MySQL later.	MySQL (needs a server), MongoDB (not relational)
Frontend	HTML + Jinja2 templates + one CSS file	Server-rendered pages; no JavaScript framework to learn; one page per feature.	React (needs Node build tools, steeper learning curve)
Charts	Chart.js 4 (one file, bundled locally)	One script tag; works offline.	D3.js (complex), Google Charts (needs internet)
Testing	pytest + Flask test client	Simple test functions; the test client drives real pages against a fresh database.	unittest, Selenium
Version control	Git + GitHub	Shared code, history as contribution evidence.	–
Diagrams	Mermaid (text-based), draw.io	Diagrams stored as text next to the code; easy to update.	StarUML

Table 12: Technology stack

Only one third-party package (Flask) is needed to run the app. pytest is used only for the tests.

10. Implementation Progress

The implementation currently has about **2,418 non-blank lines** of application code (Python, SQL, HTML templates, CSS and JS, excluding tests), 11 staff pages plus their forms, and 9 customer pages. The **end-to-end feature "meter reading → automatic bill → online payment → notification"** is fully working and covered by automated test IT-06.

10.1 Features completed

Feature	Pages	Requirement(s)
Login, logout, role-based access	/login, all pages	FR-01, FR-02
Customer registration, search and profile	Customers, Add Customer, Customer detail	FR-03, FR-04
Connections and meters	Connections, New Connection	FR-05
Meter reading entry with live validation	Meter Readings, Add Reading	FR-06, FR-07
Automatic slab billing and itemised printable bill	Billing, Bill Details	FR-08, FR-09

Online / counter / partial payments with receipts	Pay Bill, Payments	FR-10 – FR-12
Complaint management	Complaints (staff and customer), Complaint detail	FR-13
Outage management with area notifications	Outages (staff and customer), Report Outage	FR-14, FR-15
Tariff editing	Tariffs	FR-16
Employees	Employees	FR-17
Admin and customer dashboards	Dashboard (both portals)	FR-18
Customer self-service	My Connections, Usage History, Profile, Notifications	FR-20

Table 13: Completed features

10.2 Features under development

- **Reports module (partial):** area-wise and category analytics, collection efficiency and overdue list work; date-range filter and CSV export are pending.
- **Meter replacement workflow** (FR-23), **late-payment surcharge and reminders** (FR-21), **server-generated PDF invoice** (FR-22).
- **MySQL version of the schema** for the DBMS course, including a stored procedure for bill generation.
- Security hardening: CSRF tokens on forms, login rate limiting.

10.3 Screenshots of the working system

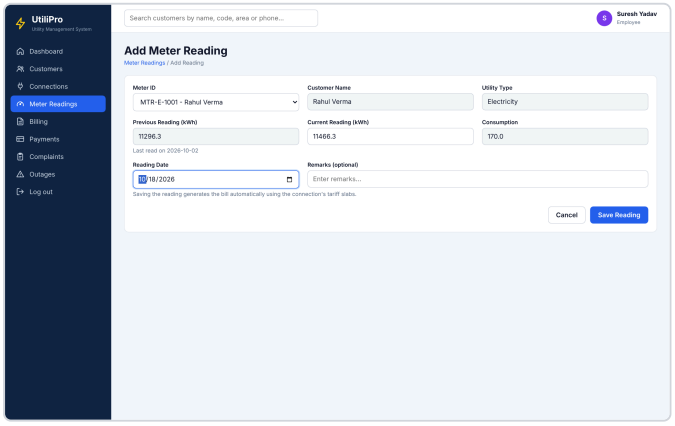


Figure 9: Add Meter Reading: previous reading loaded, consumption calculated live

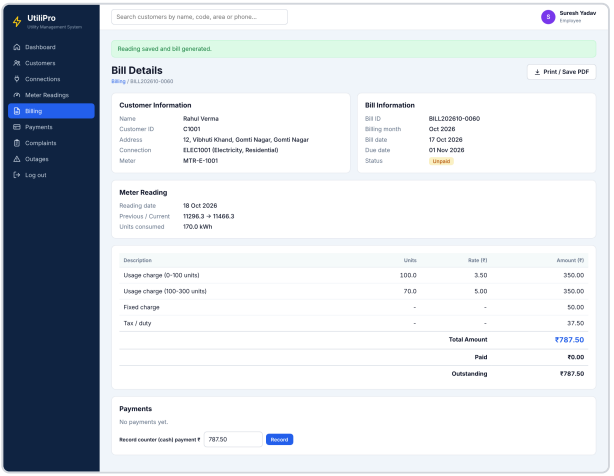


Figure 10: Bill generated automatically with slab breakdown

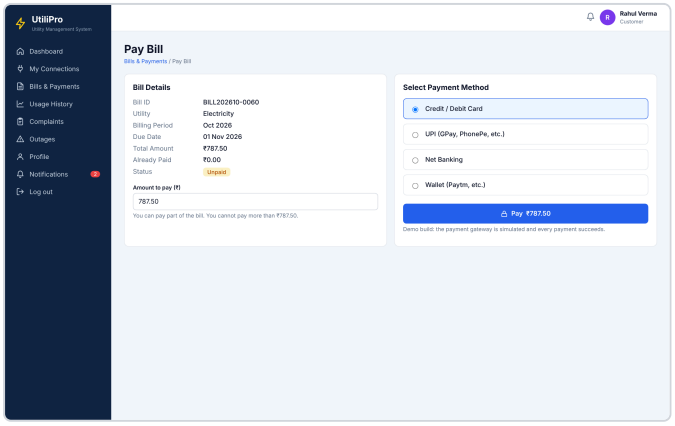


Figure 11: Customer pay-bill page

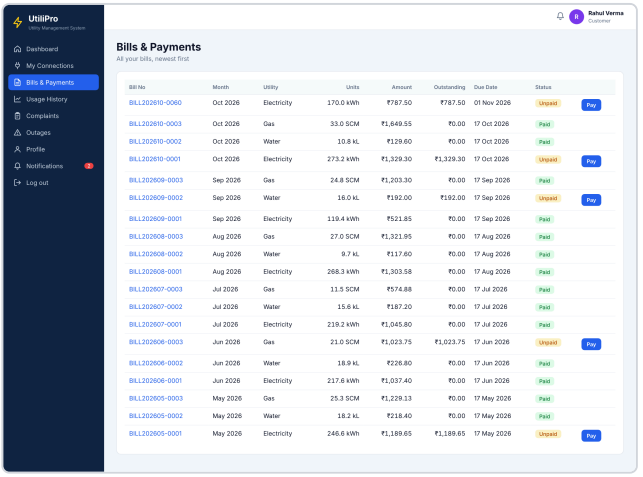


Figure 12: Bill after payment: status Paid, receipt listed

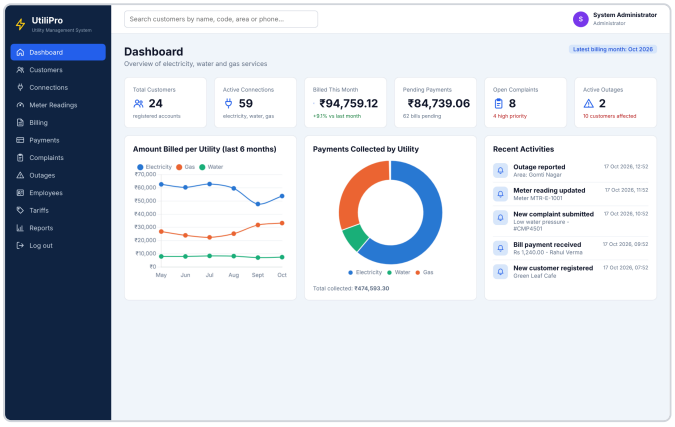


Figure 13: Admin dashboard

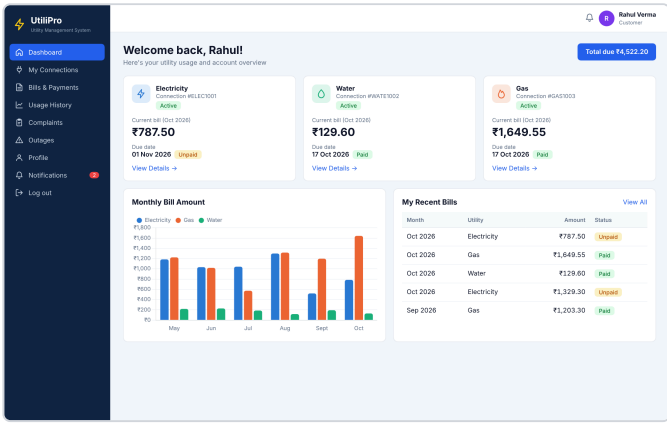


Figure 14: Customer dashboard

All other pages are shown in Appendix B.

10.4 Important implementation decisions

1. **Business rules in the database, not only in Python.** Monotonic readings, overpayment and status updates are triggers, so the rules hold even if a page has a bug or data is entered another way. The Python code catches the trigger error and shows its message to the user.
2. **Validation twice for readings.** JavaScript gives instant feedback (good usability), and the trigger gives the real guarantee (JavaScript can be bypassed).
3. **Billing logic is a pure function** (`calculate_bill`), separate from Flask and SQL, so it is easy to unit-test with hand-calculated examples.
4. **Bill snapshot.** Charges are copied onto the bill at creation time so tariff edits never change old bills.
5. **One transaction per user action.** A reading and its bill are committed together; if bill creation fails, the reading is rolled back too.
6. **Customer data isolation.** Customer pages never take a customer id from the URL; they always use the logged-in user, and bill pages check ownership and return 403.
7. **Server-rendered pages instead of a JavaScript framework**, to keep the stack simple for the team.
8. **Charts never mix units:** charts that compare utilities use rupees; usage charts are drawn separately per utility (kWh, kL, SCM).

9. **No secrets in code:** the secret key comes from `config.py` (git-ignored); a sample `config.example.py` is provided.

11. Testing Performed

Testing so far consists of **unit tests** for the billing calculation and **functional (integration) tests** that drive the real Flask application through its pages with a fresh, seeded SQLite database for each test. All tests are automated with pytest and can be re-run with `python -m pytest -v` in `04_Source_Code`. The actual results below were produced by running the suite: **20 passed in 15.21s**. Manual exploratory testing of every page was also done for each role (screenshots in Appendix B).

11.1 Test-case table

ID	Feature	Input / steps	Expected result	Actual result	Status
UT-01	Slab billing	250 kWh, residential electricity (0-100 @3.50, 100-300 @5.00), fixed 50, tax 5%	Energy 1,100.00; tax 57.50; total 1,207.50	As expected	**PASS**
UT-02	Slab billing	0 units	Only fixed + tax = 52.50, no slab rows	As expected	**PASS**
UT-03	Slab billing	Exactly 100 units (slab edge)	Only first slab used; total 420.00	As expected	**PASS**
UT-04	Slab billing	400 units (reaches 300+ slab)	Energy 2,050.00; total 2,205.00; 100 units in top slab	As expected	**PASS**
UT-05	Slab billing	-5 units	ValueError raised	As expected	**PASS**
UT-06	Slab billing	15.5 kL water, no tax	Energy 146.00; total 186.00	As expected	**PASS**
IT-01	Login	admin / admin123	Redirect to /staff/dashboard; dashboard loads	As expected	**PASS**
IT-02	Login	admin / wrong	Stays on login with 'Invalid username or password'	As expected	**PASS**
IT-03	Access control	Open /staff/bills without logging in	Redirect to /login	As expected	**PASS**
IT-04	Access control (RBAC)	Customer opens staff page; employee opens admin-only Tariffs	HTTP 403 for both; employee can open Add Reading	As expected	**PASS**
IT-05	Data isolation	Customer C1001 opens a bill of customer C1002	HTTP 403 on bill and pay page	As expected	**PASS**
IT-06	END-TO-END: reading → bill → payment	Employee enters reading +250 kWh; customer pays 1,207.50 by UPI	Bill of 1,207.50 created (unpaid); after payment status = paid and 'Payment received' notification	As expected	**PASS**

IT-07	Meter reading validation	Current reading 10 below previous	Trigger error 'Reading cannot be lower...'; no bill created	As expected	**PASS**
IT-08	Meter reading validation	Reading date 2020-01-01	Trigger error 'Reading date must be after...'	As expected	**PASS**
IT-09	Payment validation	Pay outstanding + 100	Trigger error 'Payment exceeds outstanding balance'; bill not paid	As expected	**PASS**
IT-10	Counter payment	Billing officer pays half the bill in cash	Bill status = partial	As expected	**PASS**
IT-11	Outage alerts	Admin reports electricity outage in Alambagh	Every Alambagh electricity customer notified; Hazratganj customers not notified	As expected	**PASS**
IT-12	Complaints	Customer raises complaint; technician marks it resolved	Complaint visible to staff; status resolved with timestamp	As expected	**PASS**
IT-13	Customer registration	Register with existing email	Error 'already exists'; customer count unchanged	As expected	**PASS**
IT-14	Tariff management	Admin changes slab rate to 9.99	New rate saved; old bill total unchanged	As expected	**PASS**

Table 14: Test cases with expected and actual results (UT = unit test, IT = integration/functional test)

11.2 Evidence

The full pytest log is in `05_Testing/Test_Evidence/pytest_output.txt`. The screenshots below show the same rules being enforced in the user interface.

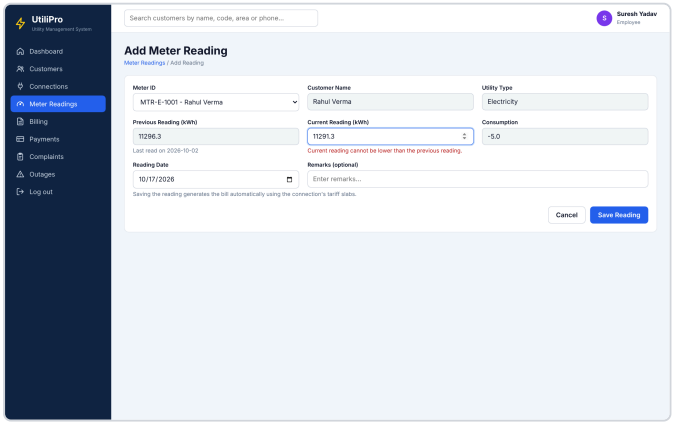


Figure 15: IT-07: lower reading blocked

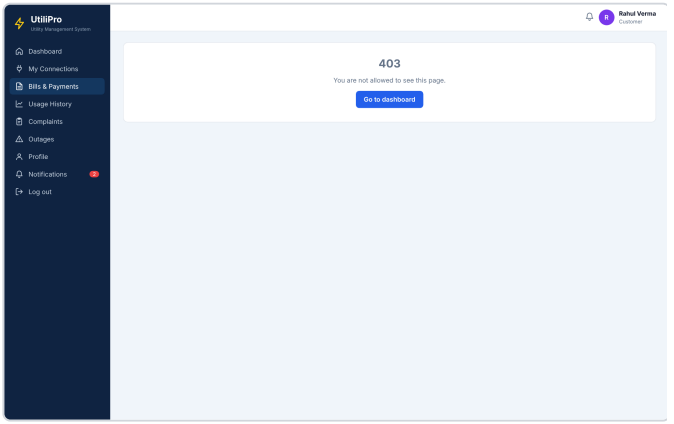


Figure 16: IT-04: customer opening a staff page gets 403

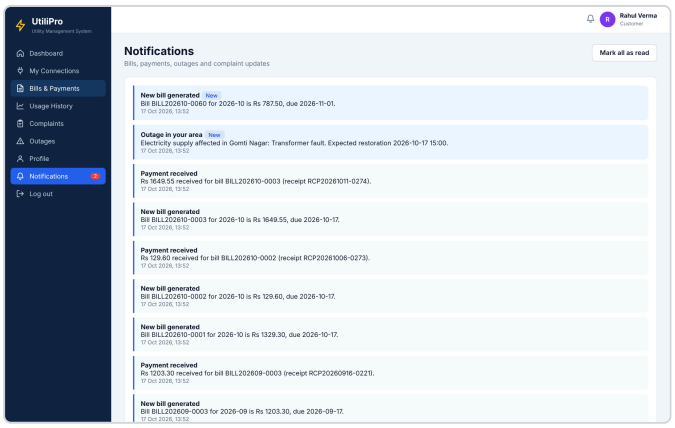


Figure 17: Trigger-generated notifications for the customer

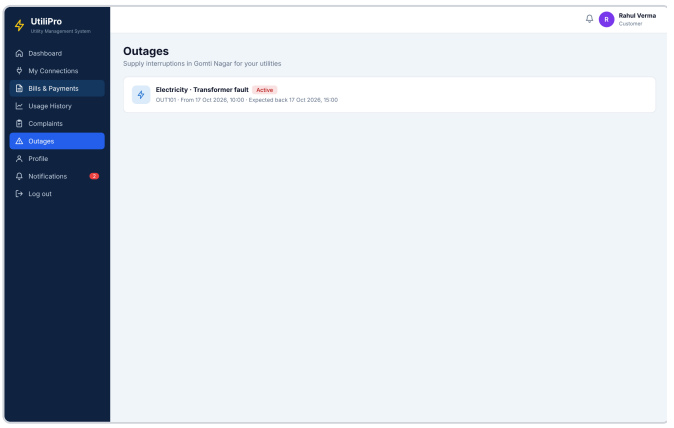


Figure 18: IT-11: outage shown to customers in that area

11.3 Testing still to be done

- System testing of the complete workflow by a person outside the team, with a written checklist.
- Boundary tests for forms (very long text, special characters) and a basic security test (SQL injection attempts; all queries are already parameterised).
- Performance check with 1,000+ customers of generated data (NFR-07).
- User-acceptance testing with 3–5 classmates acting as customers and employees.

12. Project-Management Status

12.1 Original project plan

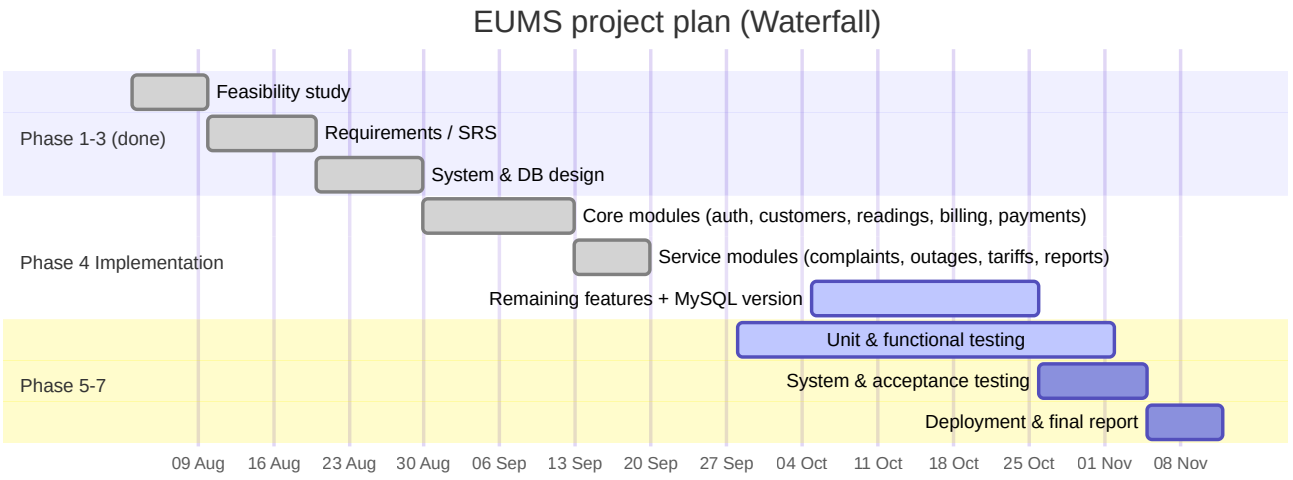


Figure 19: Project schedule (Gantt chart) following the Waterfall phases

Week	Phase	Planned activities	Deliverable
1	Feasibility	Topic study, existing systems, technical/economic/operational/schedule feasibility	Feasibility report
2–3	Requirements	Stakeholders, functional and non-functional requirements, MoSCoW, use cases, user stories	SRS v1.0

4–5	Design	Architecture, ER diagram, normalised schema, triggers, UML, UI mock-up	Design document, schema.sql
5–7	Implementation I	Auth, customers, connections, readings, billing, payments	Working core (end-to-end feature)
8	Implementation II	Complaints, outages, tariffs, dashboards, notifications	Mid-semester build
8–9	Testing I	Unit and functional tests for completed modules	Test report v1
9–11	Implementation III	Reports, meter replacement, late fees, PDF invoice, MySQL port, CSRF	Feature-complete build
12–13	Testing II	System testing, UAT, performance test, bug fixing	Final test report
14	Deployment	Installation guide, final report, demo	Final submission

Table 15: Week-wise plan

12.2 Effort and schedule estimation (Basic COCOMO)

The current build has 2,418 lines of code, so the finished system is estimated at about **3.5 KLOC**. It is a small project built by a small team with familiar requirements, so we use the **organic** mode (a = 2.4, b = 1.05, c = 2.5, d = 0.38):

$$\text{Effort } E = 2.4 \times (3.5)^{1.05} = \mathbf{8.9\ person\text{-}months} \quad \text{Duration } D = 2.5 \times E^{0.38} = \mathbf{5.7\ months} \quad \text{Staff} = E / D \approx \mathbf{1.6\ full\text{-}time\ people}$$

Four students working part-time (about 6 hours a week each, roughly 0.15 of a full-time person) give about 0.6 full-time equivalents over 3.5 months, which is about 2 person-months. That is much less than the COCOMO estimate. The gap is expected: COCOMO was calibrated on industrial projects that include full documentation, formal reviews and maintenance, and it overestimates very small projects that use a high-level framework. We use it as an upper bound and manage the gap through MoSCoW scoping (Should/Could features can be dropped without breaking the system).

12.3 Current progress

Phase / work package	Planned by mid-sem	Actual	Status
Feasibility study	100%	100%	On track
Requirements (SRS)	100%	100%	On track
System and database design	100%	100%	On track
Implementation	50%	~60% (19 of 25 requirements, 1 partial)	Ahead
Testing	20%	20 automated tests, all passing	On track
MySQL version for DBMS course	Started	Not started	Delayed
Deployment and documentation	–	README and run instructions done	Later

Table 16: Progress against plan

12.4 Delayed or pending tasks

- **MySQL schema port** – delayed because the team first chose SQLite to simplify setup; moved to weeks 9–10.
- **Reports date-range filter and CSV export** – pending (Could priority).
- **Meter replacement, late fees, PDF invoice** – planned for weeks 9–11.
- **CSRF protection** – identified during design review; scheduled before final submission.

12.5 Risks and mitigation plan

#	Risk	Prob.	Impact	Mitigation
R1	Team members unfamiliar with Flask / web development	High	High	Chose the simplest stack; one sample page used as a pattern for all others; pair programming.
R2	Wrong bills because of calculation bugs	Medium	High	Billing as a pure function with hand-checked unit tests; bills store their own breakdown.
R3	Invalid data entered (lower readings, overpayment)	High	High	Database triggers and constraints plus UI validation; covered by tests IT-07 to IT-09.
R4	Customer sees another customer's data	Low	High	Ownership checks on every customer page; test IT-05.
R5	Late discovery of requirement errors (Waterfall weakness)	Medium	Medium	Review at the end of every phase; early end-to-end feature; MoSCoW scope buffer.
R6	Schedule clash with exams and other courses	High	Medium	Must features finished first; weekly meeting; tasks re-assigned when someone is busy.
R7	Losing code or conflicting edits	Medium	High	Git/GitHub with one branch per feature; frequent commits.
R8	Member unavailable near deadline	Medium	Medium	Shared documentation and code walkthroughs, so anyone can explain any module.

Table 17: Risk register

13. Individual Contribution

13.1 Individual contribution table

#	Name	Reg. no.	Responsibilities and work done	Share
1	Sushrut Lad	_____	Team lead, backend (billing, readings), database triggers	25%
2	Member 2	_____	Database design, ER model, seed data, reports module	25%
3	Member 3	_____	Frontend pages, customer portal, UI design	25%
4	Member 4	_____	Requirements/SRS, testing, documentation	25%

Table 18: Individual contributions

13.2 Task-allocation table

Task	Phase	Responsible	Support	Status
Feasibility study	Feasibility	Sushrut Lad	Member 4	Done
Requirements and SRS	Requirements	Member 4	Sushrut Lad	Done
Use cases and user stories	Requirements	Member 4	Member 3	Done
Architecture and module design	Design	Sushrut Lad	Member 2	Done
ER diagram, schema, normalisation	Design	Member 2	Sushrut Lad	Done
Triggers and views	Design	Sushrut Lad	Member 2	Done
UI design and templates	Design / Impl.	Member 3	Member 4	Done
Auth, customers, connections	Implementation	Sushrut Lad	Member 3	Done
Meter readings and billing	Implementation	Sushrut Lad	Member 2	Done
Payments and customer portal	Implementation	Member 3	Sushrut Lad	Done
Complaints and outages	Implementation	Member 3	Member 2	Done
Dashboards and reports	Implementation	Member 2	Member 3	In progress
Seed data and sample CSV	Implementation	Member 2	Member 4	Done
Unit and functional tests	Testing	Member 4	Sushrut Lad	Done
Mid-sem report and diagrams	Documentation	Member 4	Member 2	Done
Demo video	Documentation	Member 3	Sushrut Lad	Pending
MySQL port	Implementation	Member 2	Sushrut Lad	Pending

Table 19: Task allocation

13.3 Git commit history

The repository history (`git log --oneline --all`) is the evidence of individual work. Each member commits their own module from their own GitHub account. The log is attached in `06_Project_Management/git_log.txt`.

13.4 Contribution declaration

We confirm that the contributions listed above are accurate and that every member can explain the parts of the system they worked on.

Name	Reg. no.	Signature	Date
Sushrut Lad	_____		
Member 2	_____		
Member 3	_____		
Member 4	_____		

14. Challenges Faced and Solutions Attempted

Challenge	Solution
Most of the team had never built a web application.	Picked Flask + server-rendered HTML instead of React; built one complete page together as a template, then copied the pattern for every other page.
Slab billing was hard to get right by hand (boundaries, open-ended last slab, decimals).	Wrote the formula as a small pure function, checked it with hand-calculated cases at slab edges and with decimal units (UT-01 to UT-06).
Making sure readings never go backwards, even if someone bypasses the form.	Two layers: instant JavaScript check in the form, and BEFORE INSERT triggers in the database that abort the transaction.
Old bills changing when the admin edits a tariff.	Store a snapshot of charges and bill items on each bill (deliberate denormalisation, explained in the DBMS design).
Showing electricity (kWh), water (kL) and gas (SCM) in one chart was misleading.	Comparison charts use rupees; usage history shows one chart per utility in its own unit.
Realistic demo data for six months.	Seed script reads customers from a CSV and generates readings with seasonal variation, then produces bills and payments using the real billing code.
Tests interfering with each other through a shared database.	Each test gets a fresh database file in a temporary folder.
Doing both the DBMS and SEPM projects without double work.	One code base and one schema; the DBMS report focuses on schema, normalisation and SQL objects, and this report focuses on process and engineering.

15. Work Planned for the Remaining Semester

Week	Work	Requirement
9	Meter replacement workflow; CSRF protection on all forms	FR-23, NFR security
9–10	MySQL 8 version of the schema with stored procedure `generate_bill` (DBMS course)	DBMS
10	Late-payment surcharge and overdue reminders (notifications)	FR-21
11	Reports: date-range filter and CSV export; server-generated PDF invoice	FR-19, FR-22, FR-24
12	System testing, boundary and security tests, performance test with 1,000+ customers	NFR-07
13	User-acceptance testing with classmates; bug fixes	–
14	Deployment guide, user manual, final report and final demo	–

Table 20: Remaining work

16. References

1. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill, 2019.
2. I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2015.
3. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
4. B. W. Boehm, *Software Engineering Economics*, Prentice-Hall, 1981 (COCOMO model).
5. A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
6. Flask documentation, Pallets Projects – https://flask.palletsprojects.com/
7. SQLite documentation: CREATE TRIGGER, CREATE VIEW – https://www.sqlite.org/lang.html
8. Chart.js documentation – https://www.chartjs.org/docs/
9. pytest documentation – https://docs.pytest.org/
10. Uttar Pradesh Electricity Regulatory Commission, domestic (LMV-1) and commercial (LMV-2) tariff schedules (used as the model for the slab structure; rates in the system are illustrative).
11. Digital Personal Data Protection Act, 2023, Government of India.

Appendix A – AI-Tool Usage Disclosure

Tool	Purpose	Nature of generated content	Verification / modification by students
Claude (Anthropic), via Claude Code	Help with code, documentation, diagrams and test cases	Draft Flask routes, HTML templates and CSS; SQL schema with triggers and views; seed script; pytest tests; Mermaid diagram sources; draft text of this report and the SRS.	The team reviewed and ran all code, checked bill amounts by hand, confirmed every test passes, edited the report text, and filled in team-specific details. *(Team: describe your own checks and changes here.)*

All members remain responsible for the correctness of the submitted work and can explain every module.

Appendix B – Screenshots of All Pages

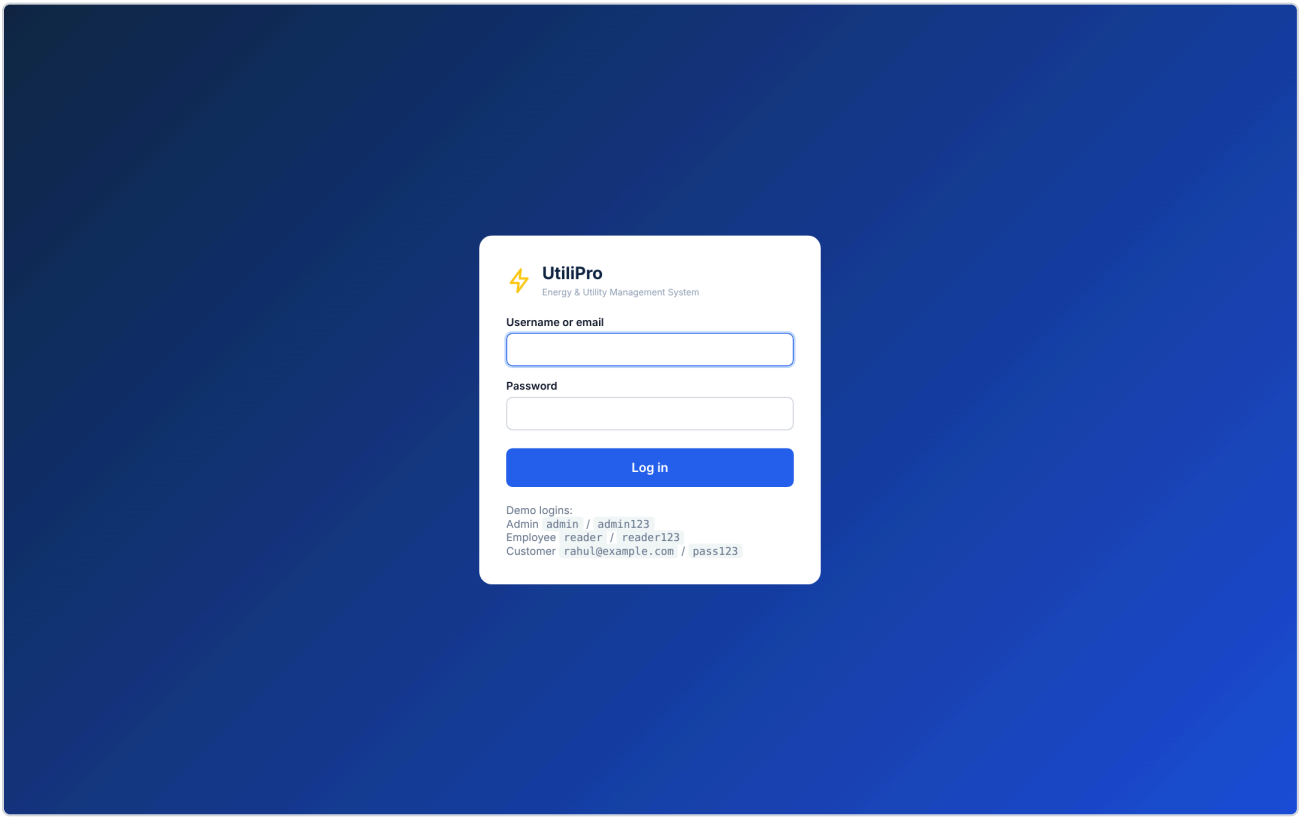


Figure 20: Login page with demo accounts

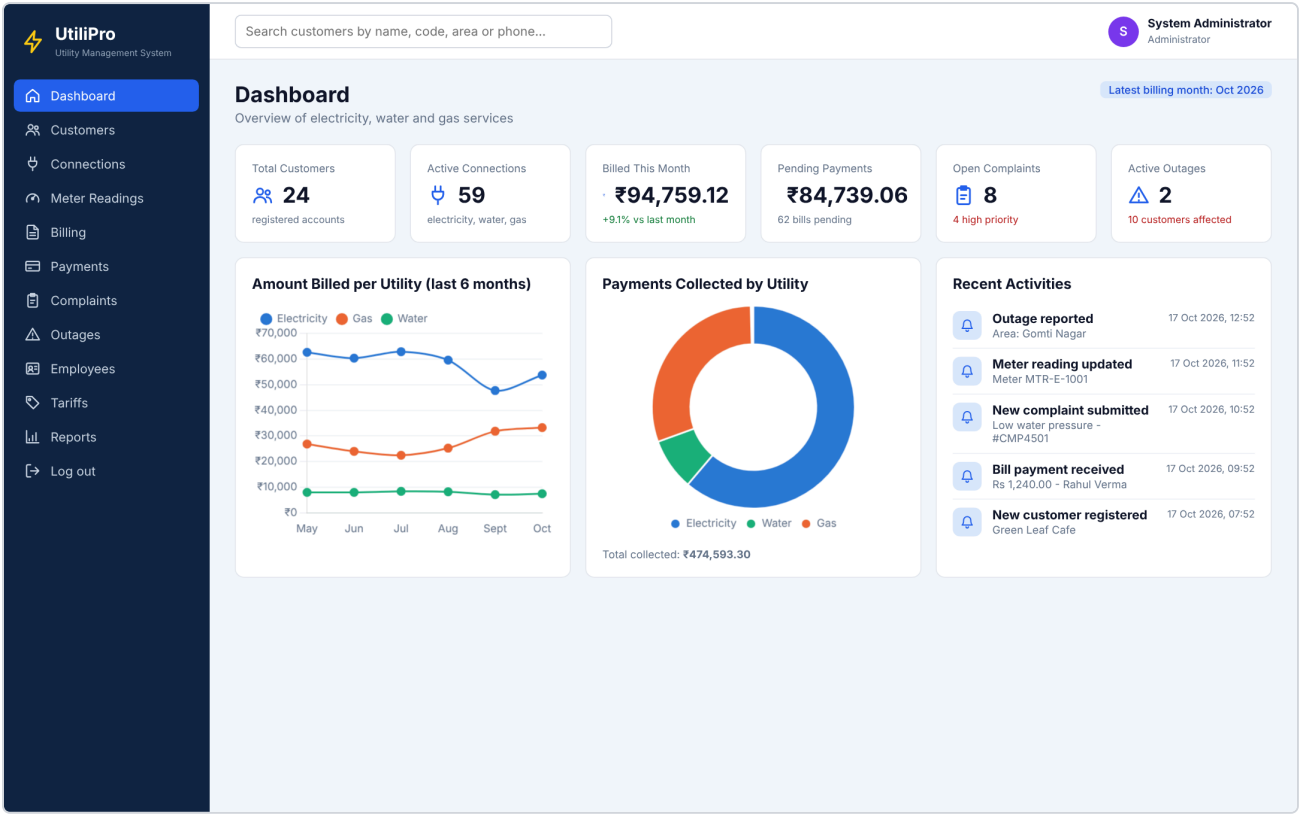


Figure 21: Admin dashboard: KPIs, billed amount per utility, collections, recent activity

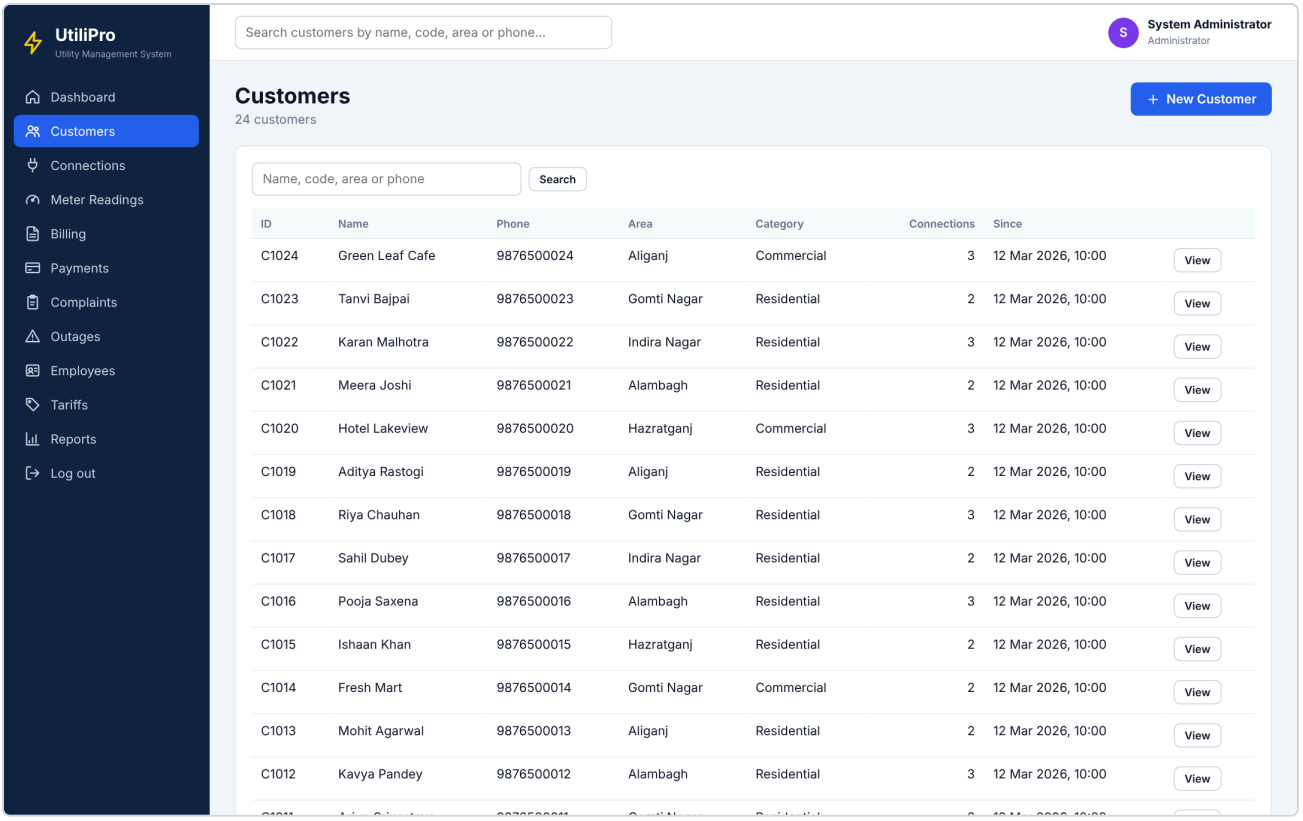


Figure 22: Customers list with search

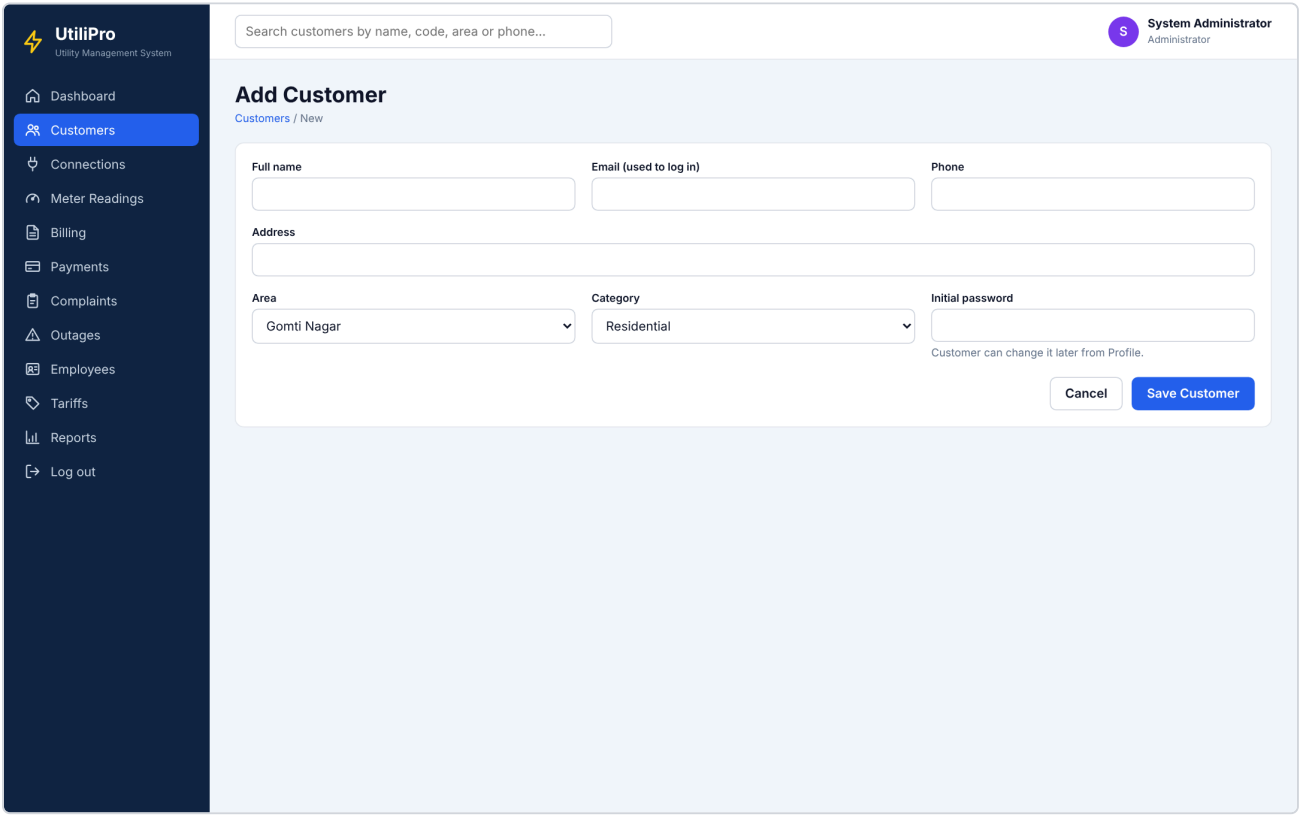


Figure 23: Add customer form

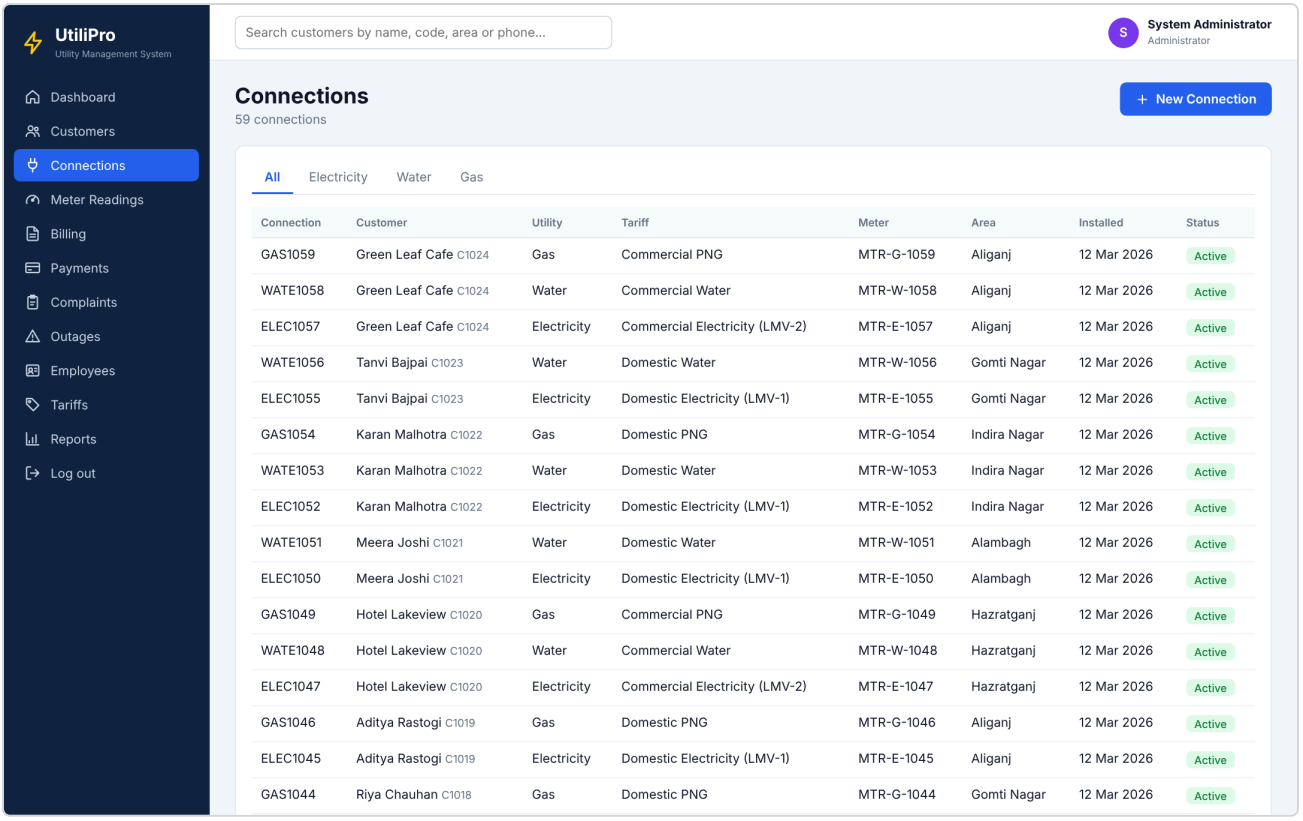


Figure 24: Connections, filterable by utility

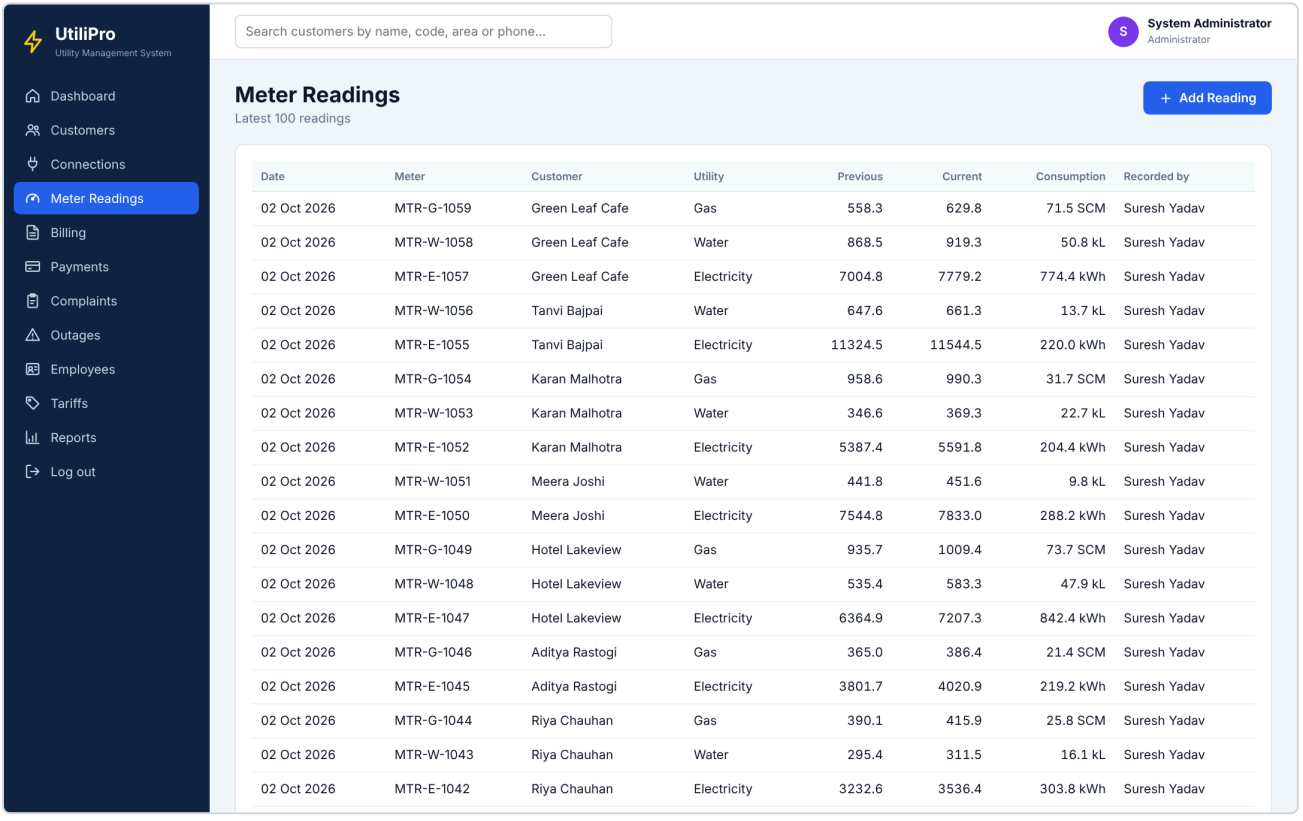


Figure 25: Meter readings list

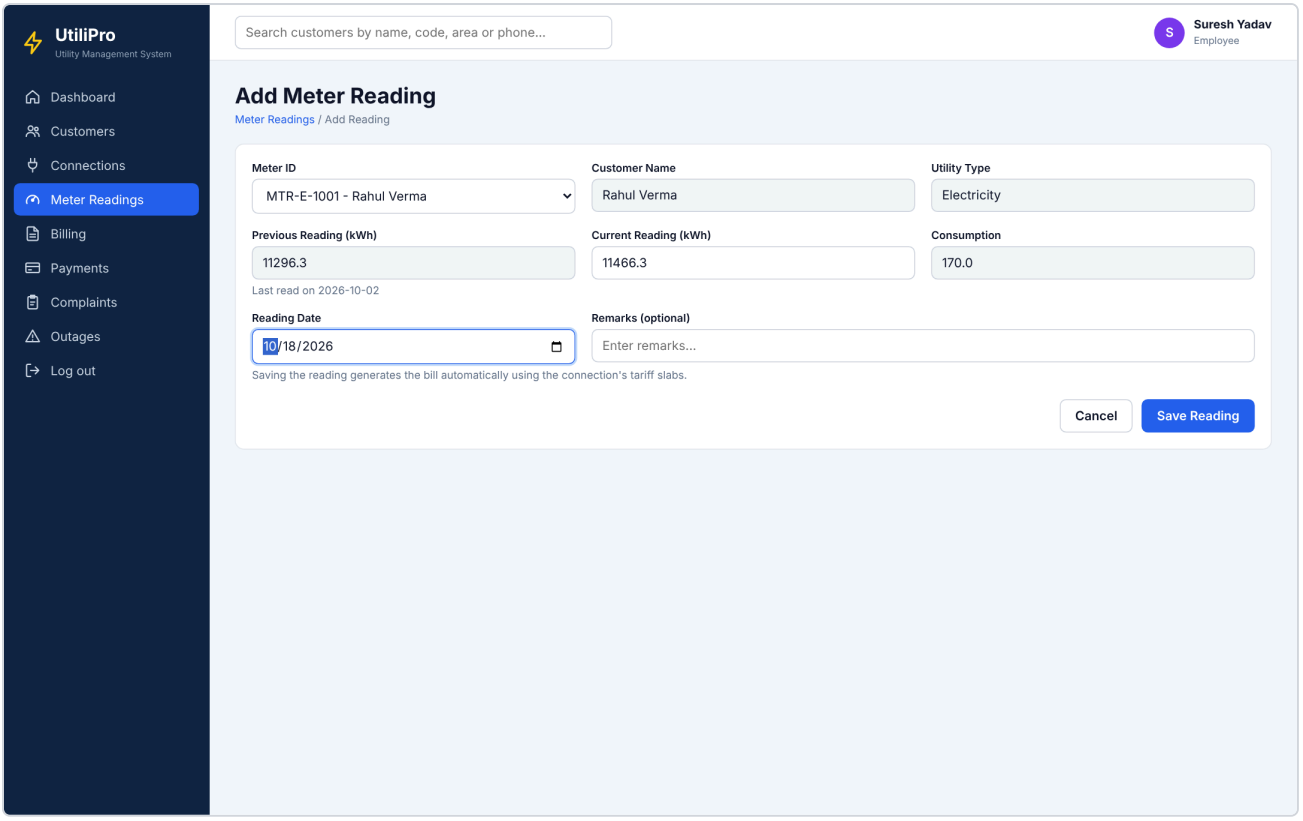


Figure 26: Add meter reading (employee)

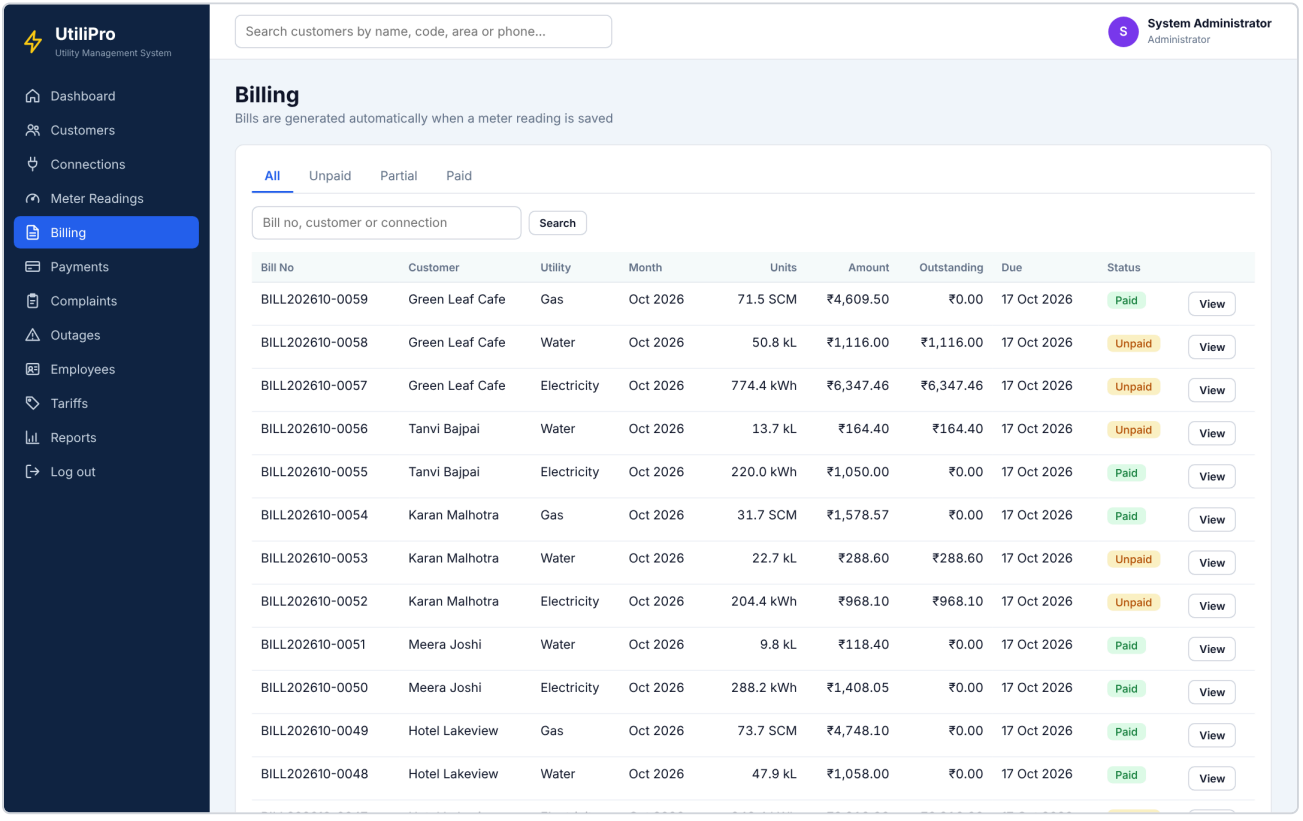


Figure 27: Billing list with status tabs

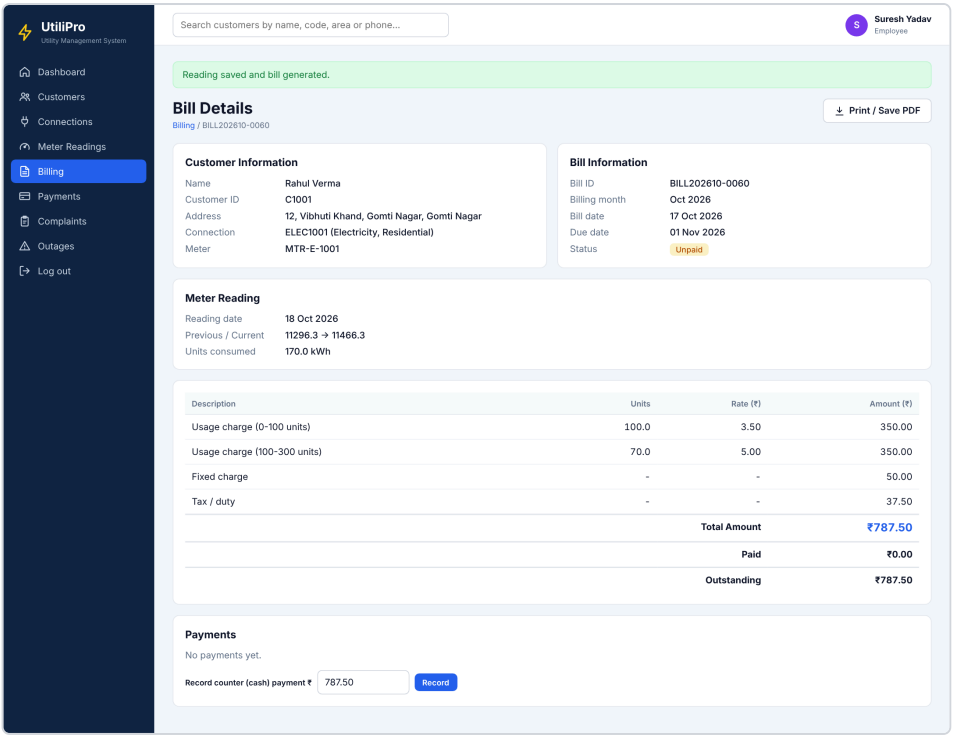


Figure 28: Bill details with slab breakdown and counter payment

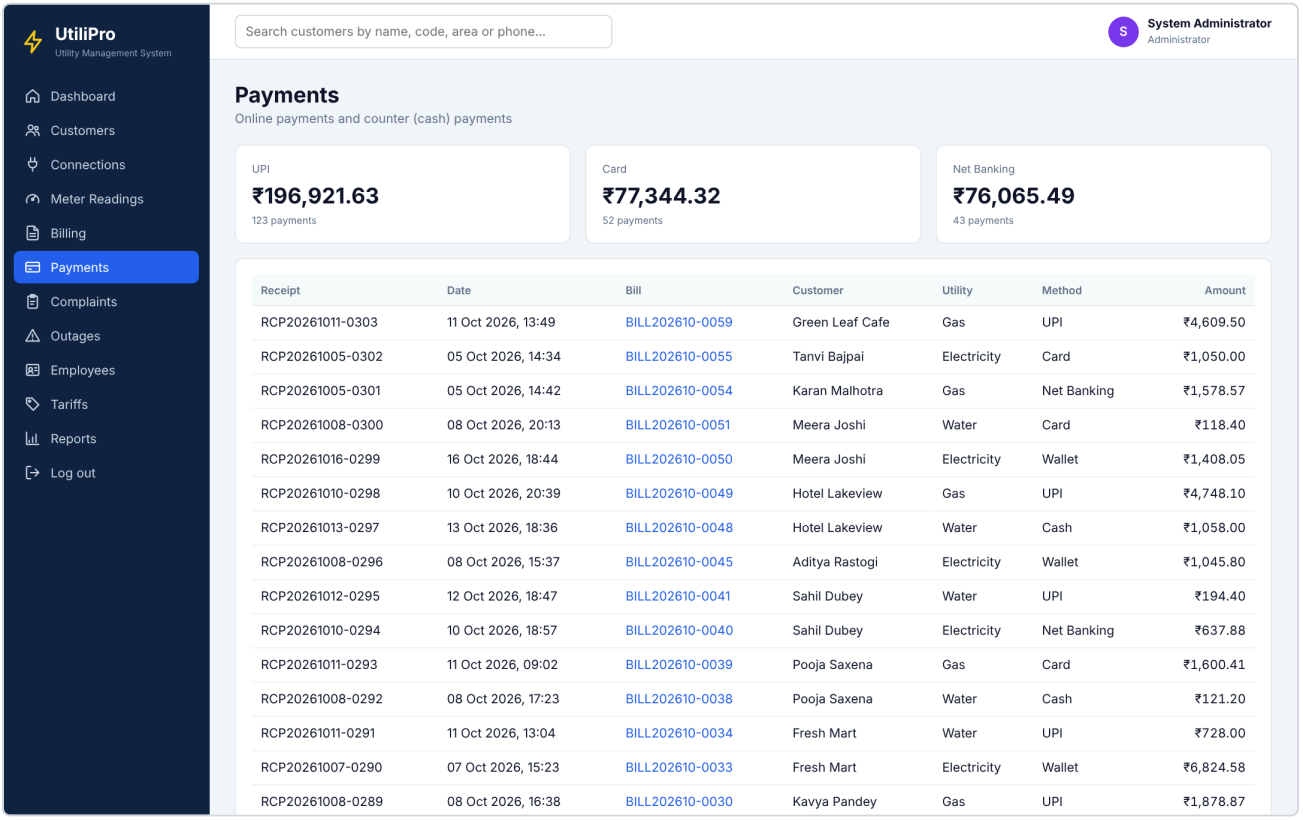


Figure 29: Payments with totals by method

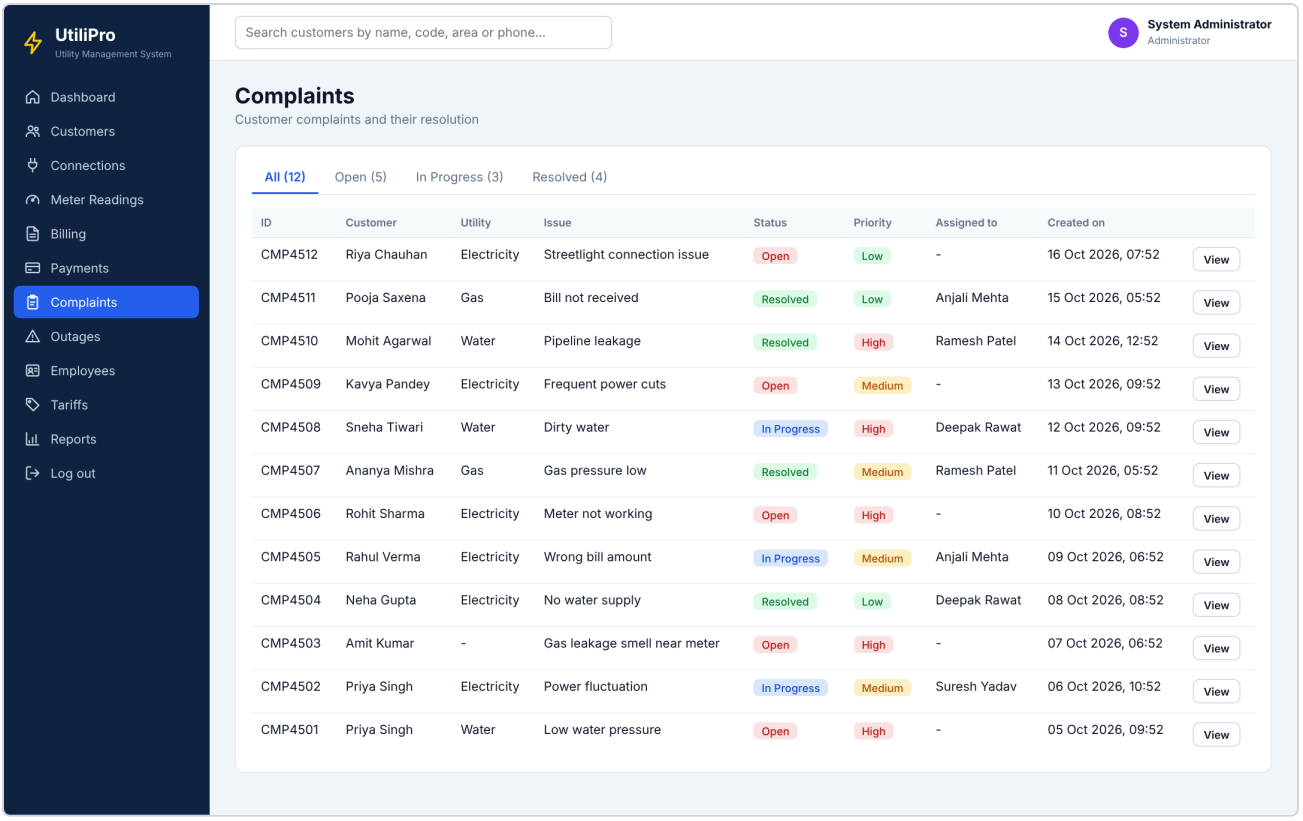


Figure 30: Complaint management

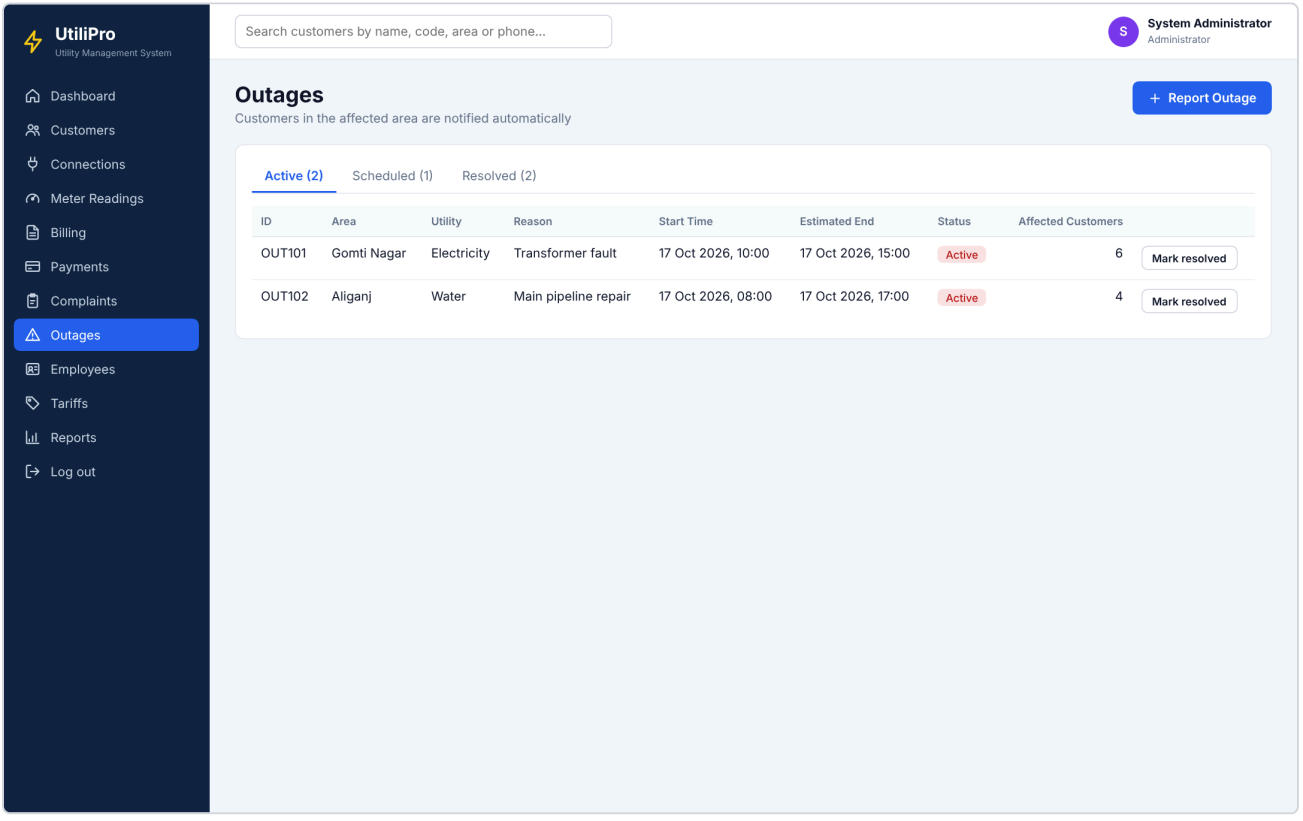


Figure 31: Outage management

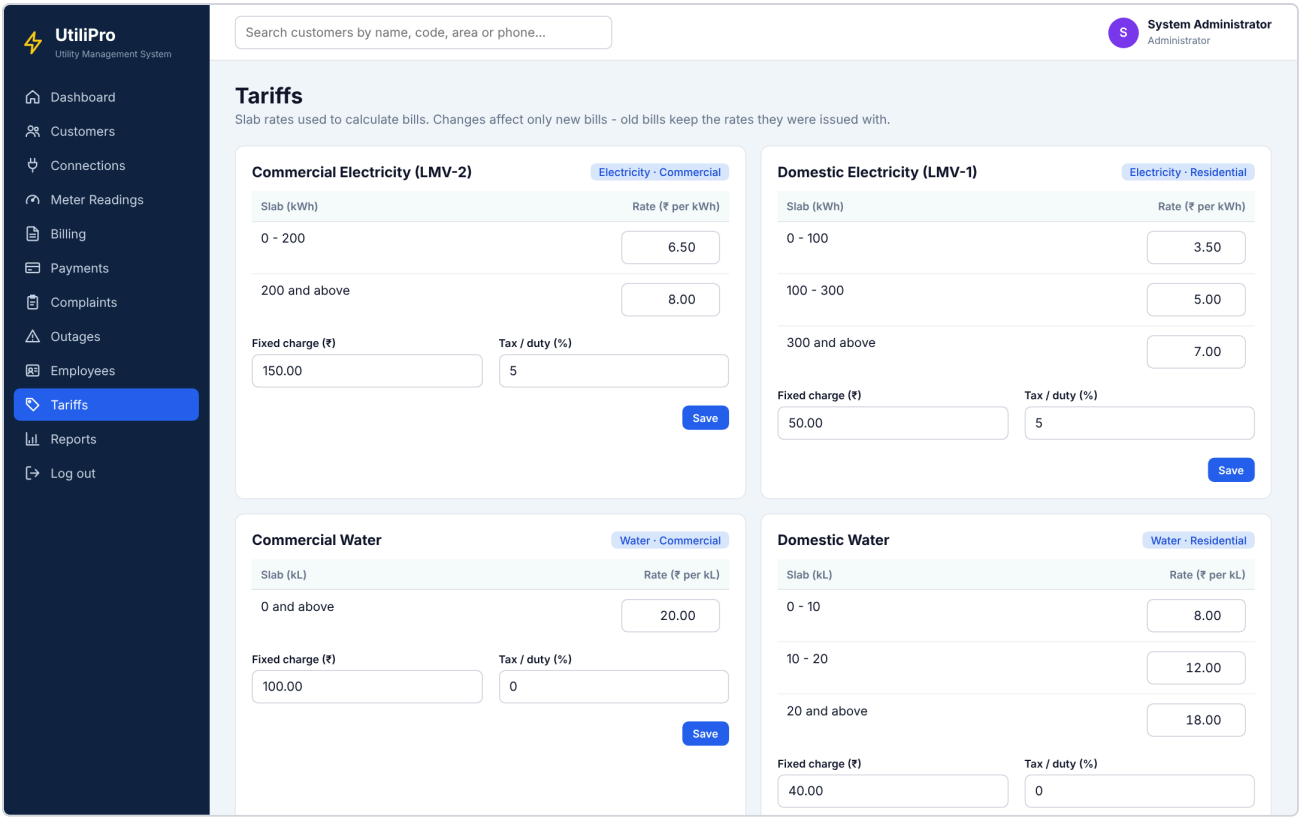


Figure 32: Tariff editor (admin)

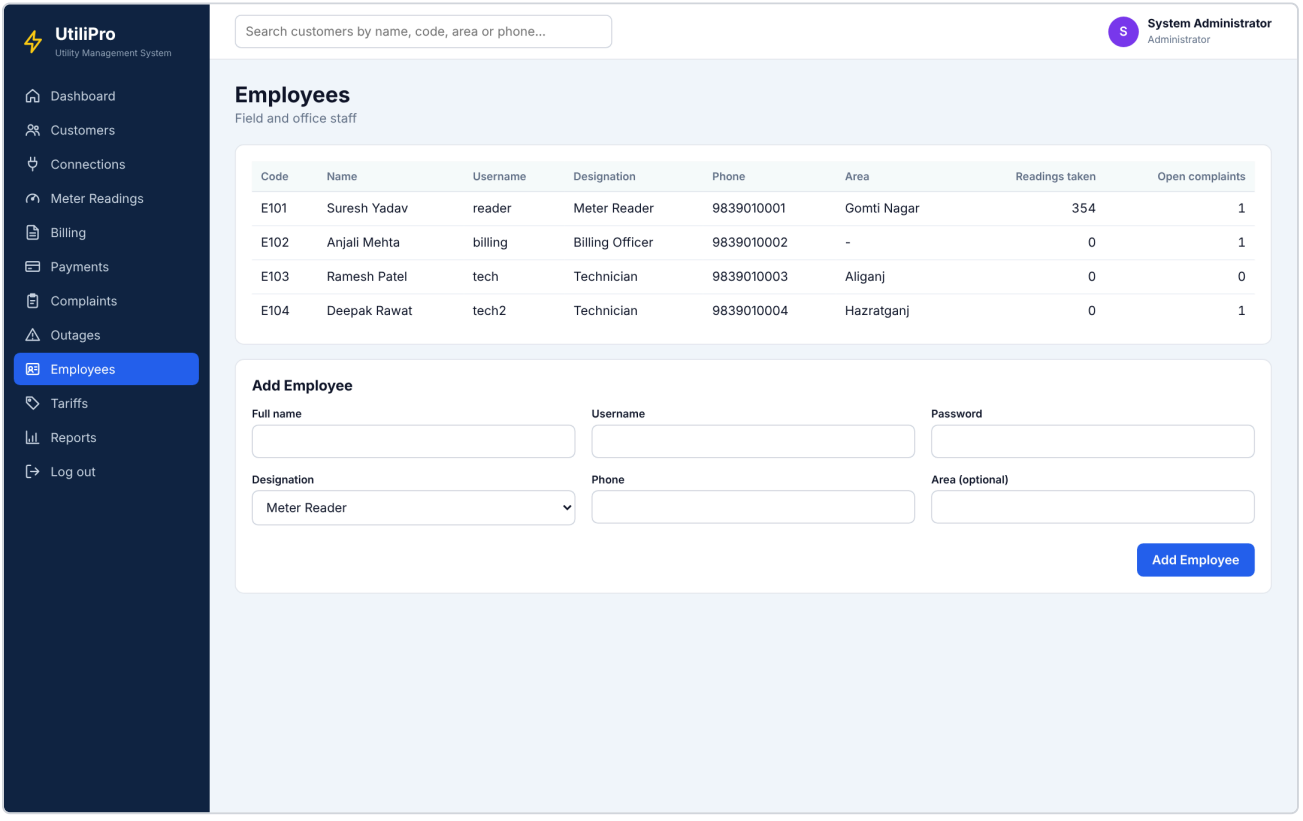


Figure 33: Employees (admin)

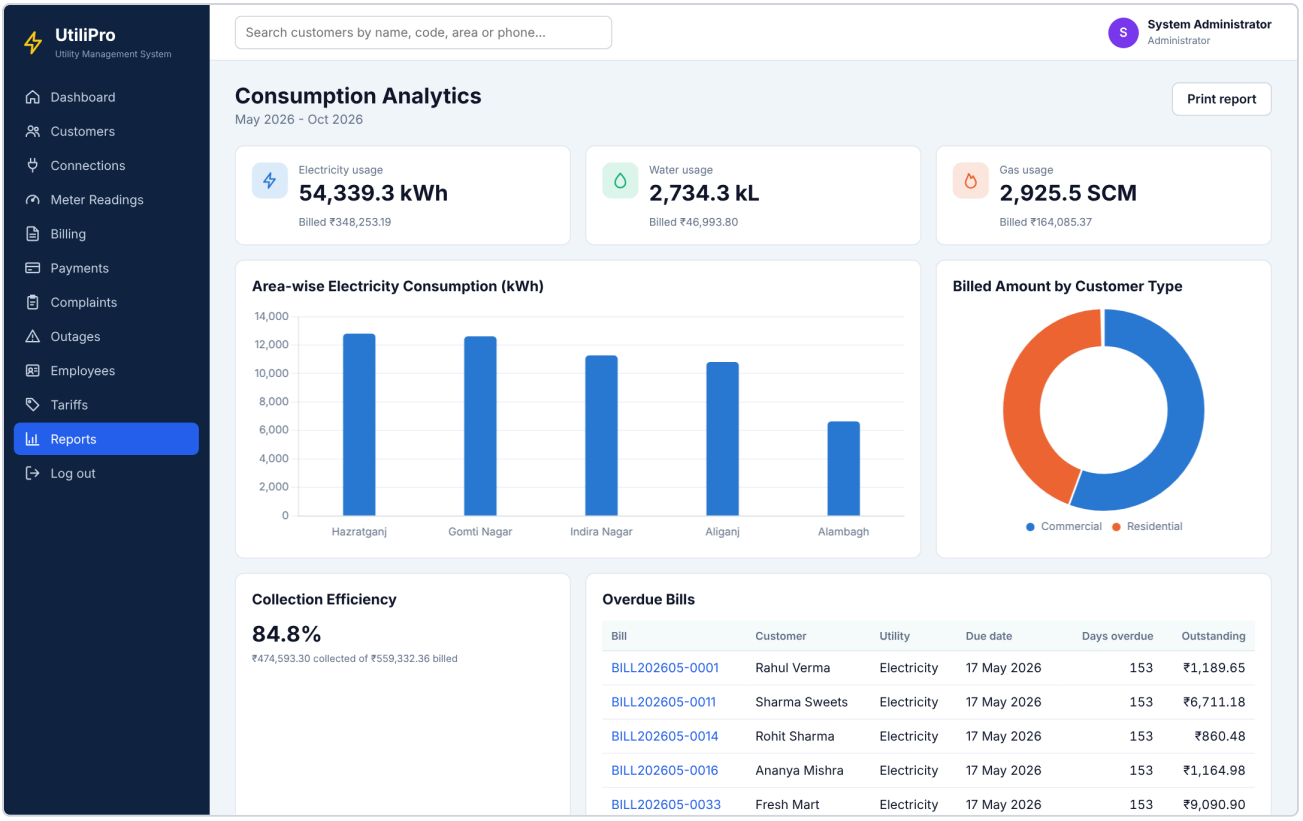


Figure 34: Consumption analytics and overdue bills (admin)

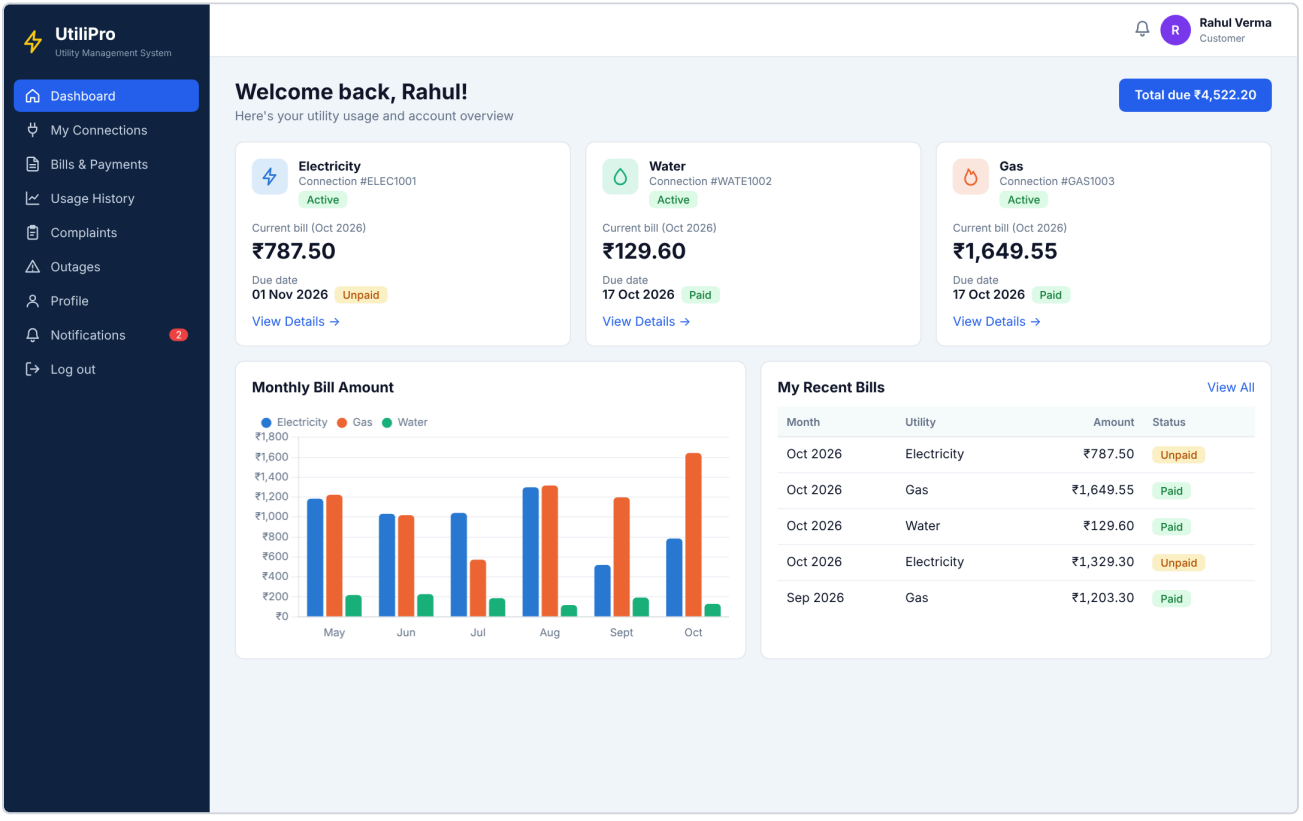


Figure 35: Customer dashboard

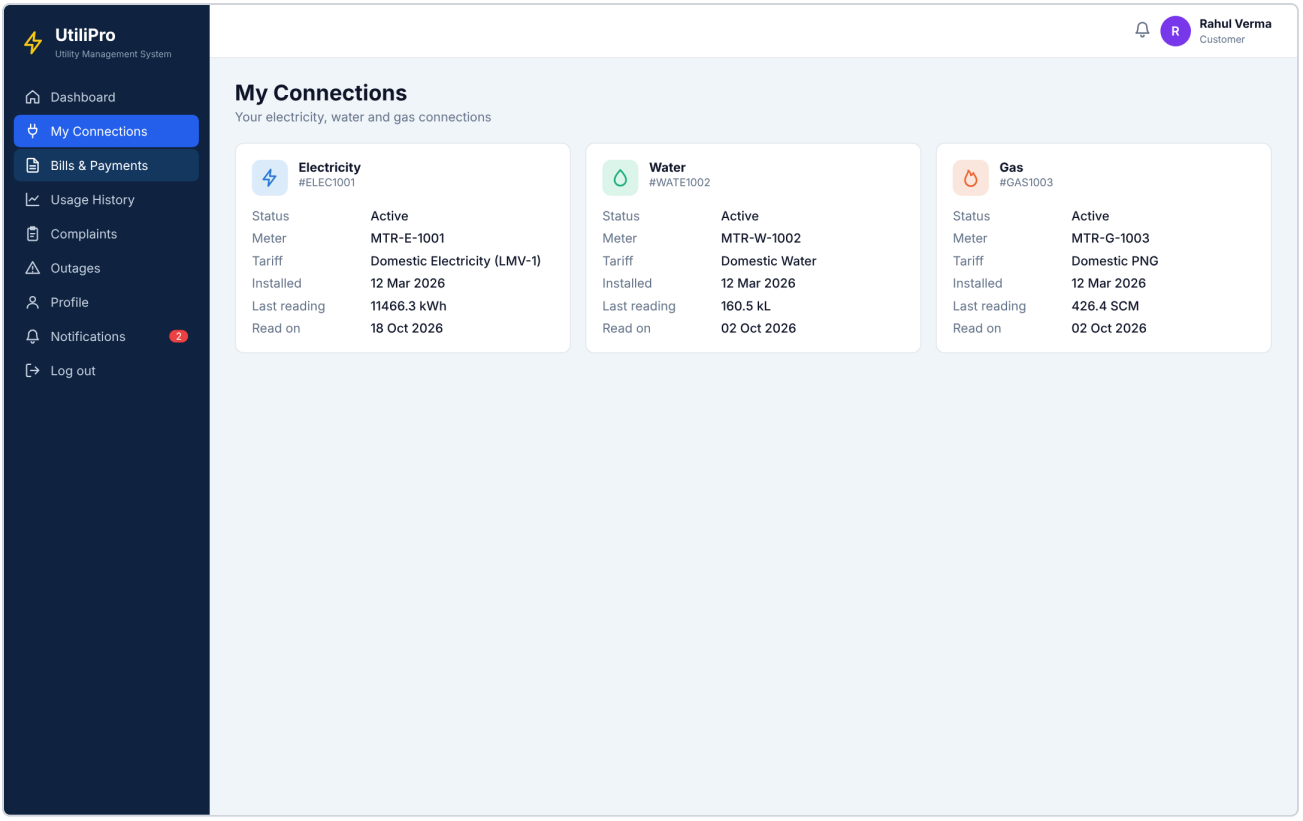


Figure 36: Customer: my connections

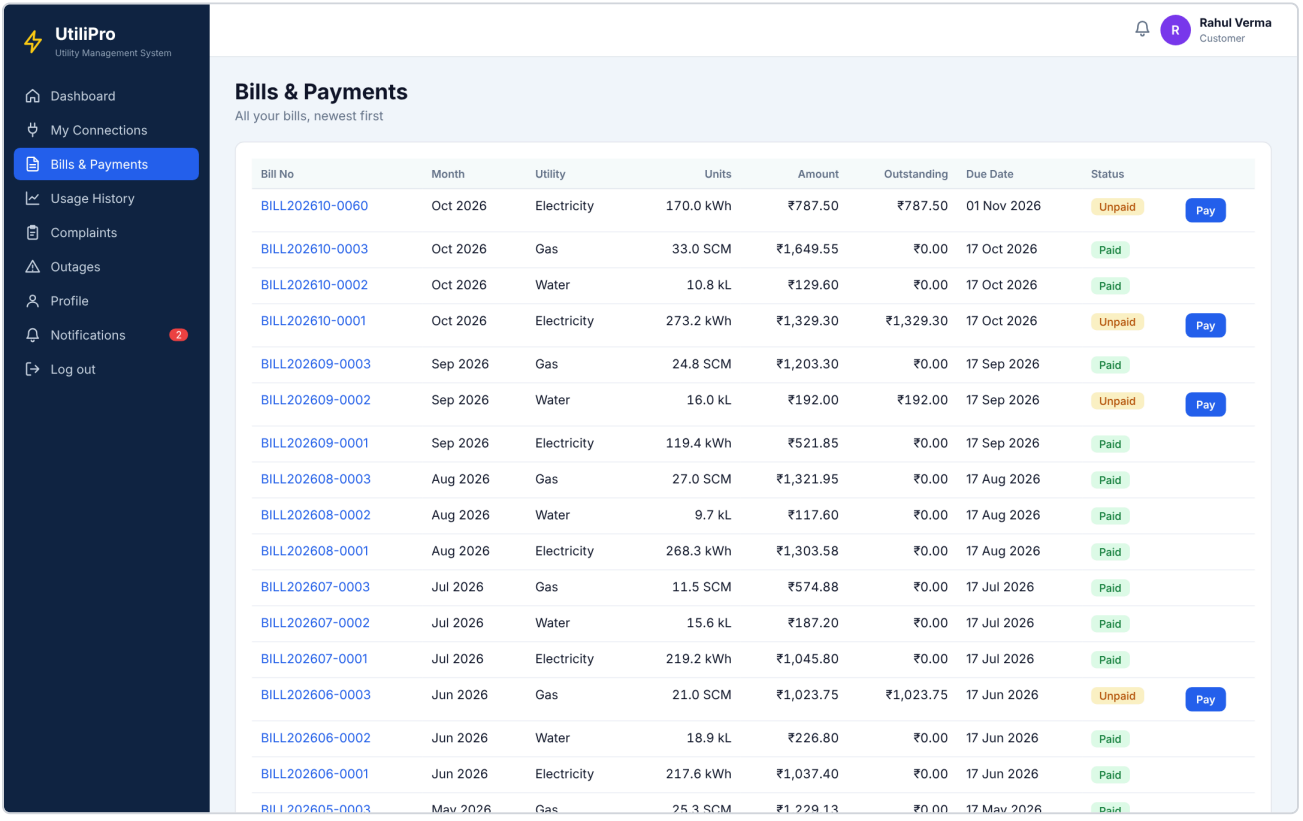


Figure 37: Customer: bills and payments

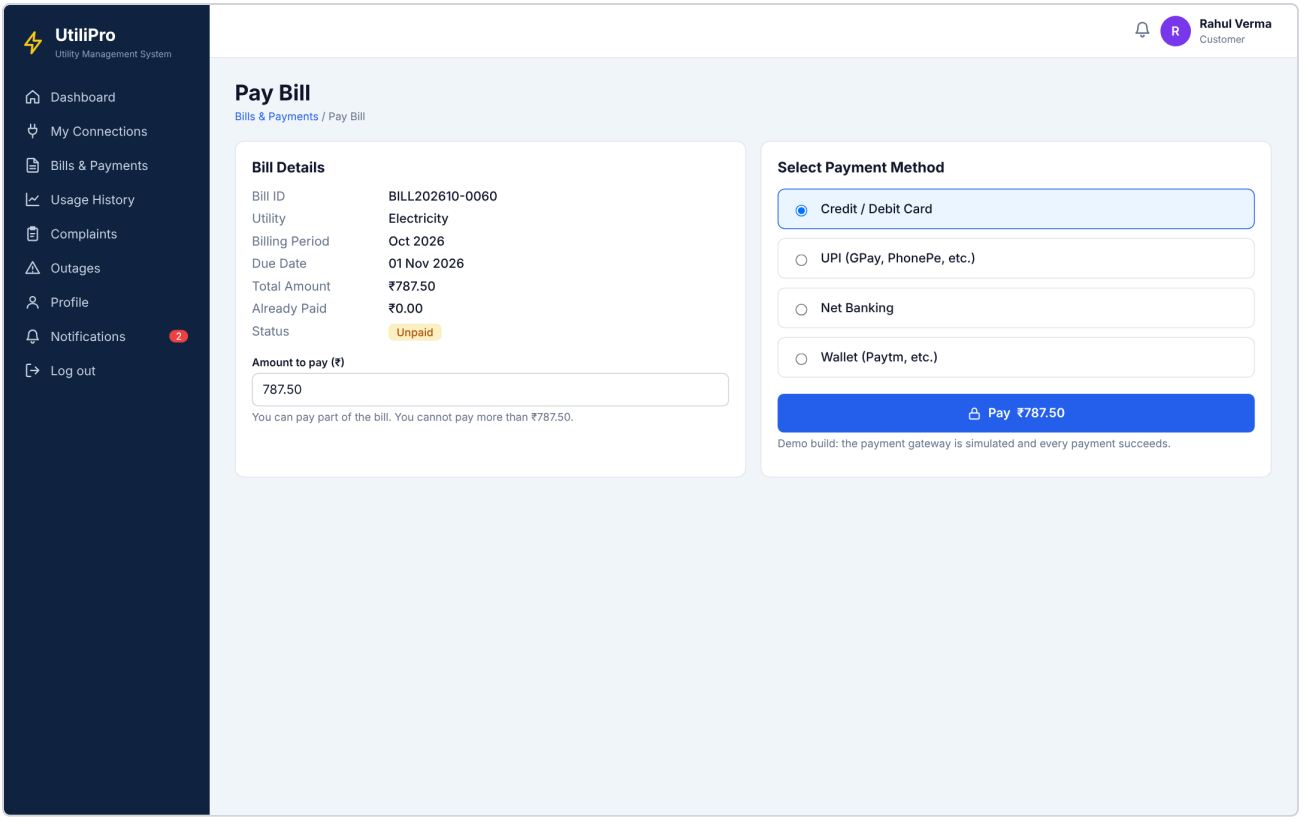


Figure 38: Customer: pay bill

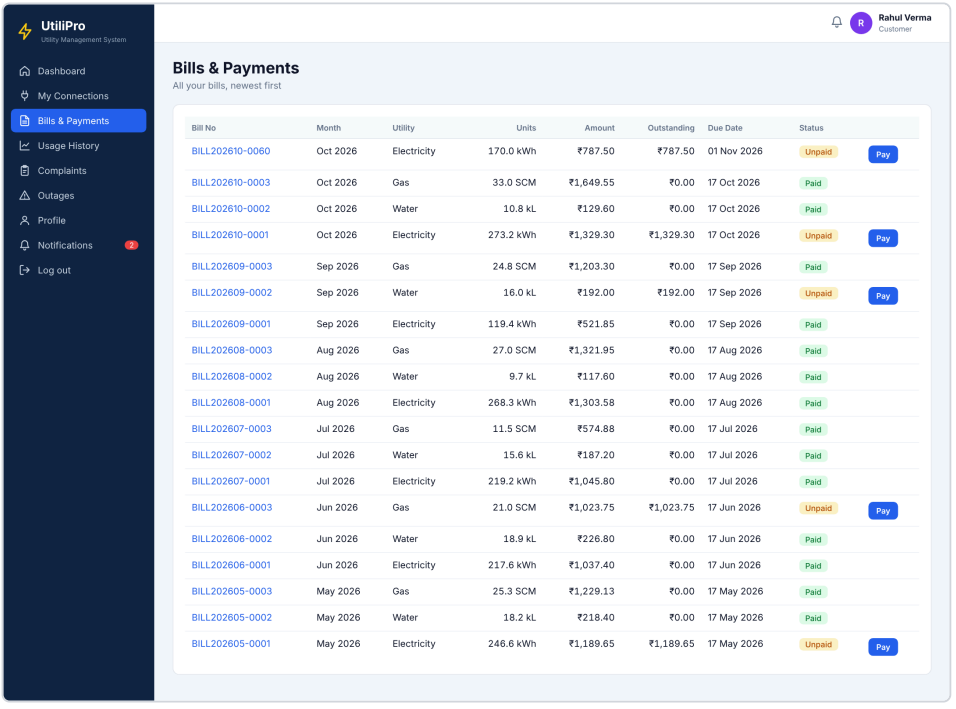


Figure 39: Customer: bill after successful payment

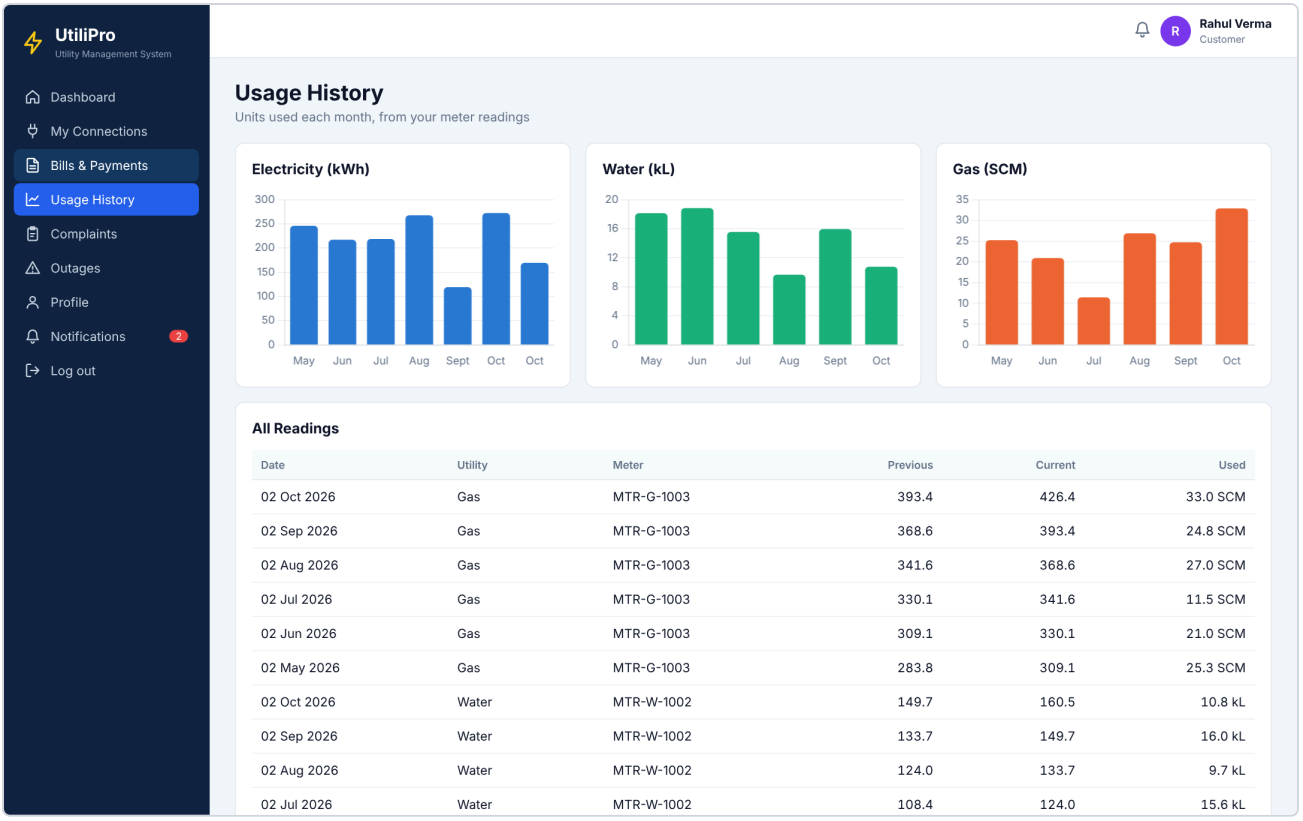


Figure 40: Customer: usage history per utility

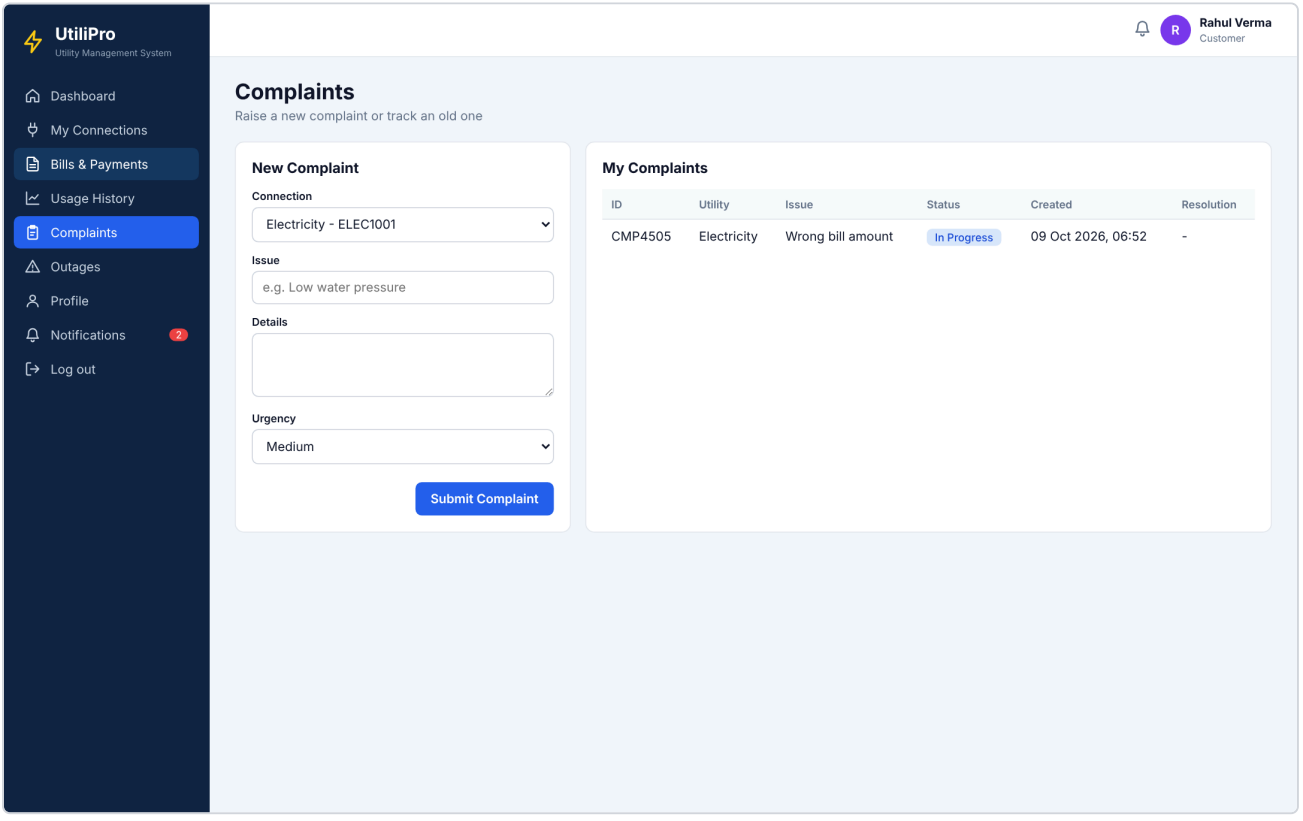


Figure 41: Customer: raise and track complaints

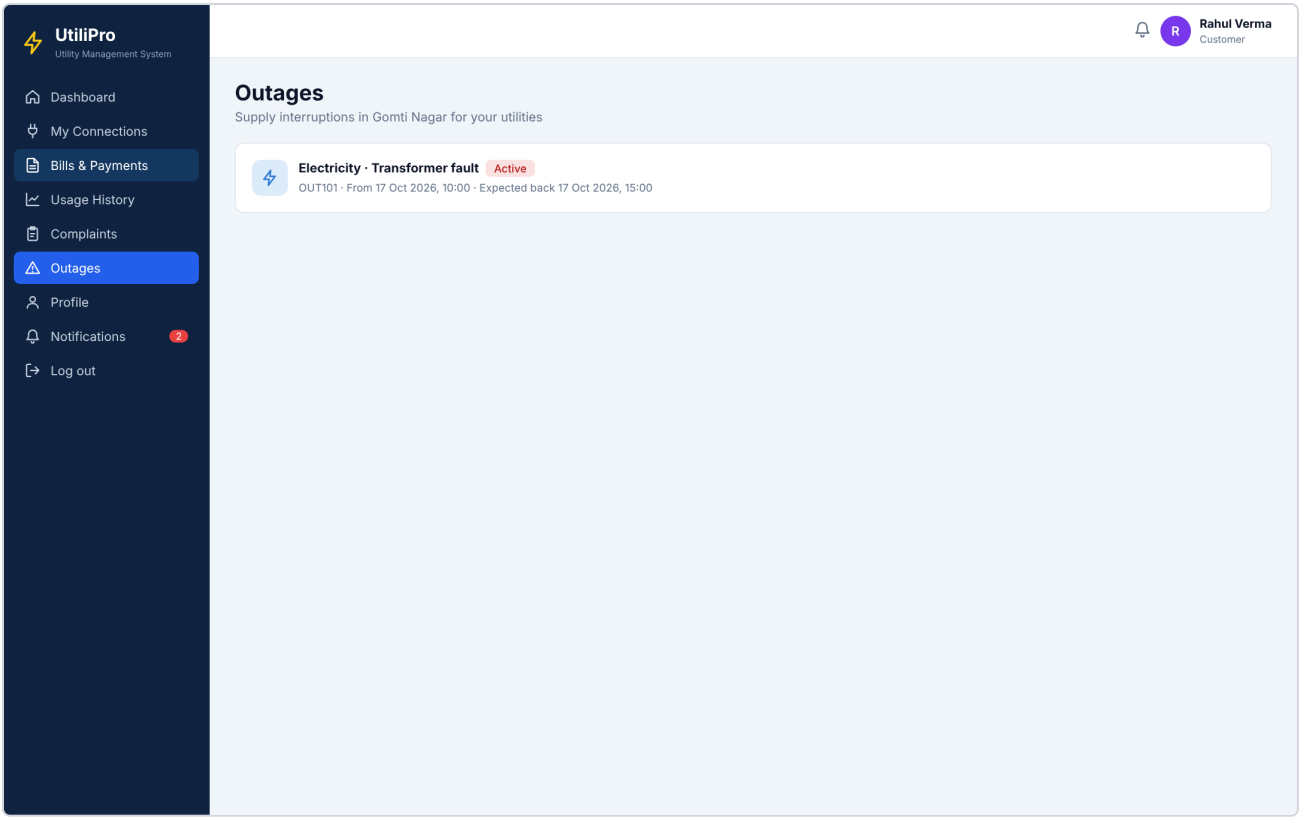


Figure 42: Customer: outages in my area

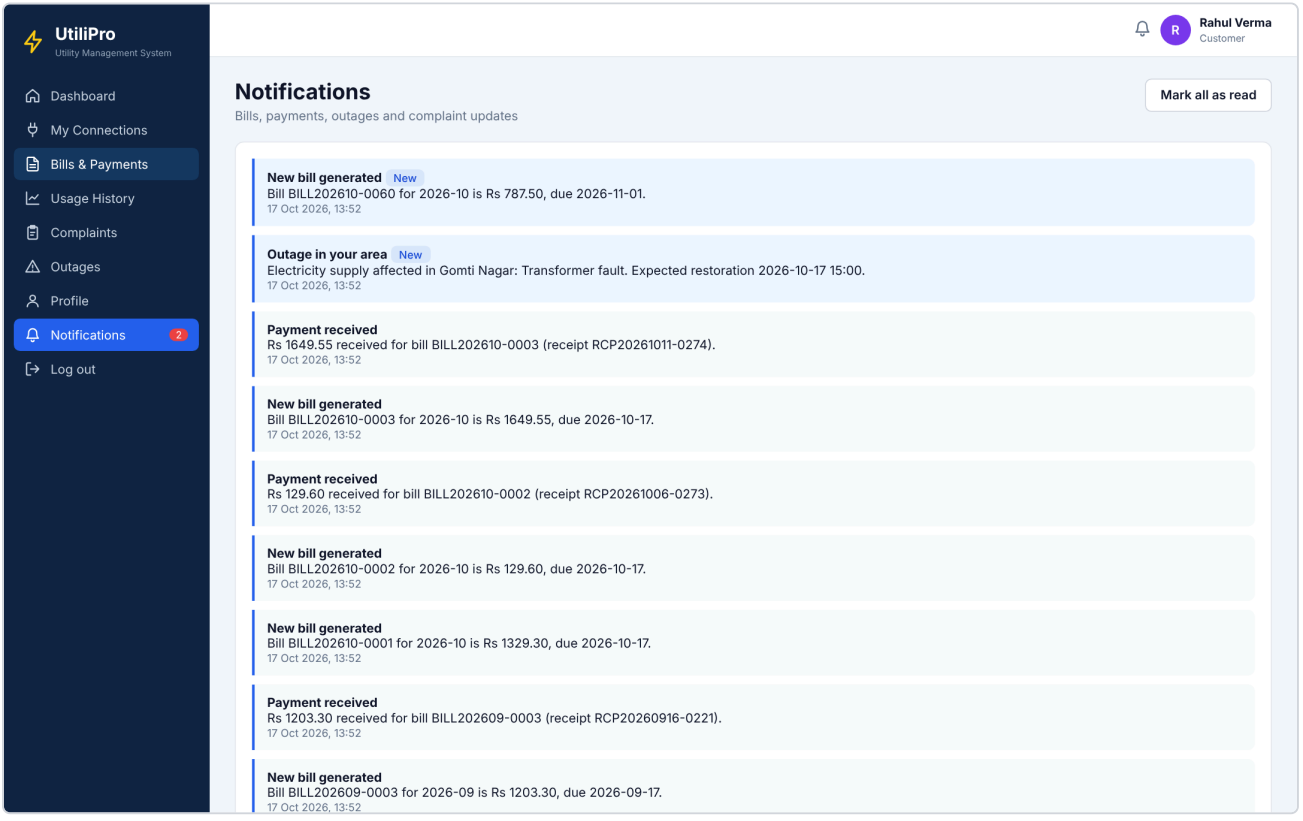


Figure 43: Customer: notifications

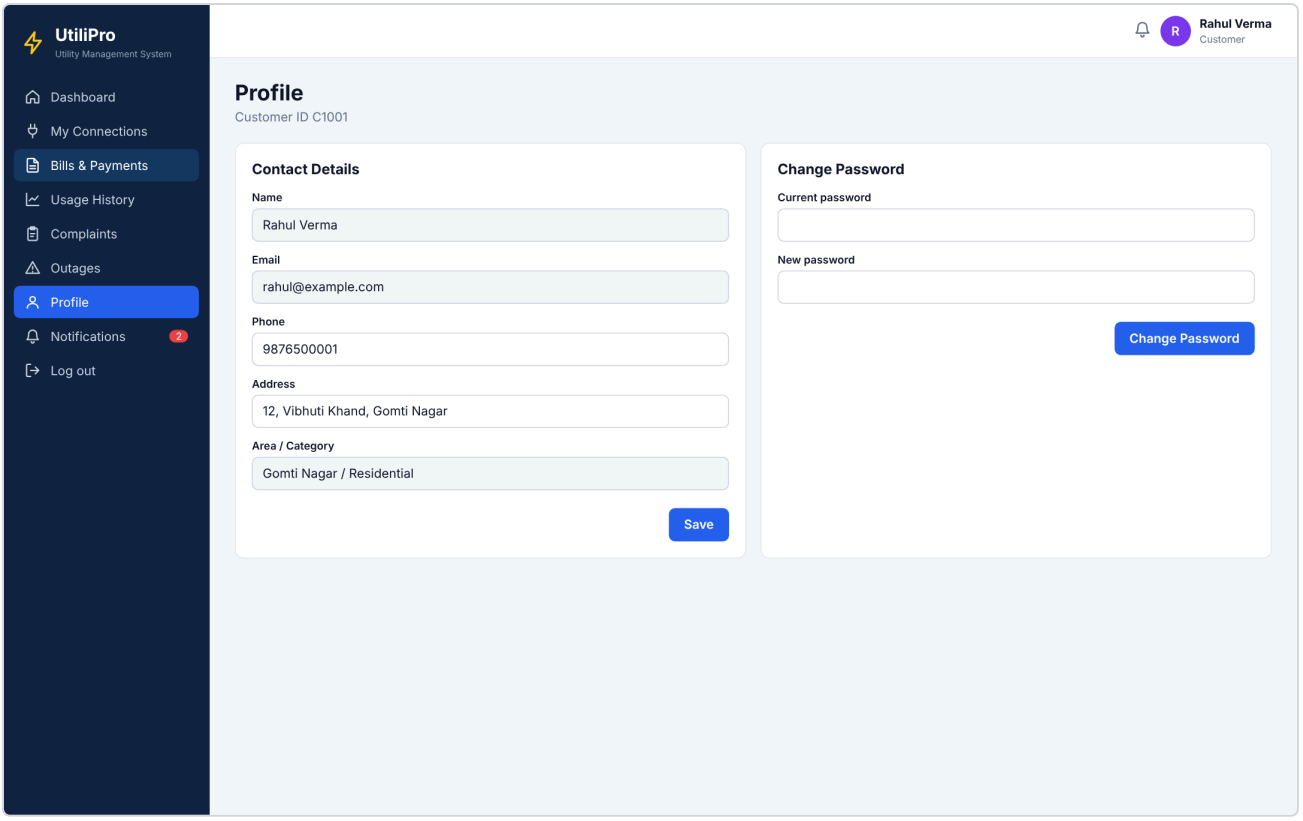


Figure 44: Customer: profile and password

Appendix C – How to Run

```
cd 04_Source_Code
pip install -r requirements.txt        # installs Flask (and pytest for tests)
python seed.py                         # creates eums.db with demo data
python app.py                          # open http://127.0.0.1:5000
python -m pytest -v                    # run the 20 automated tests
```

Role	Username	Password
Admin	admin	admin123
Employee (meter reader)	reader	reader123
Employee (billing officer)	billing	billing123
Employee (technician)	tech	tech123
Customer	rahul@example.com (and the others in sample_data/customers.csv)	pass123

Table 21: Demo accounts (sample data only)